

Instruction Manual For

PlainFlightController V3.x.Dev

Revision History

Version	Comments	Date
V06-08	Added Spektrum SRXL2 Rx protocol	16/10/2026
V06-07	Web GUI updates and minor fixes.	6/9/2026
V06-06	Addition of IBus Rx protocol	17/8/2026
V06-05	Removed IMU reversing as it is now performed by reverse servo.	27/7/2026
V06-04	Corrected description of ESP32_SoftWire library.	20/7/2026
V06-03	Review corrections.	14/7/2026
V06-02	GPS, pass through channels and other updates to align with code base.	9/7/2026
V06-00	Multiple updates for CRSF, Board type, SoftSerial, Configurator, Config and extended servo travel.	4/6/2026
V05-00	V2.1 updates, neopixel LED for Wavershare Zero/Tiny ESP32's, added new IMU orientation.	14/12/2025
V04-00	V2.0 updates for C++ and many new features.	1/3/2025
V03-00	Updates for web server interface and advanced rudder mixer.	23/6/2024
V02-00	Updates for new levelled mode, battery monitor, ESC calibration and LED outputs.	12/2/2024
V01-02	Initial release.	24/1/2024
V00-05	Development	15/12/2023

Contents

1. Introduction 4
1.1 Background 4
1.2 PlainFlightController V3.x.dev Development Release 4
1.2.1 Channel Pass Through 4
1.2.2 CRSF Rx Protocol 4
1.2.3 IBus Rx Protocol 4
1.2.4 GPS 4
1.2.5 Web Configurator 4
1.2.6 Board Type Selection 4
1.2.7 Board Orientation 4
1.2.8 Extended Servo Travel 5
1.2.9 SoftWire Library 5
1.2.10 Config Layout 5
1.2.11 Web Gui Update ..... **Error! Bookmark not defined.**
1.2.12 General Updates 5
1.3 Specifications 5
1.4 Scope 6
2. How To Assemble 6
2.1 Components 8
3. PCB's 8
3.1 WSMC 8
3.2 WSMC Mini 9
4. Arduino IDE Installation 9
4.1 Arduino IDE Installation 9
4.2 Arduino Library Installation 10
4.2.1 Espressif Systems ESP32 Installation 10
4.2.2 ArduinoJson 13
4.2.3 ESP32_SoftWire 13
4.2.4 dualGNSS GPS Library 13
4.3 ESP32-S3 Board Selection 14
4.4 IDE ESP32 Tools Configuration 15
4.4.1 USB CDC On Boot 15
4.4.2 Other Tool Configurations 16
4.4.3 Optimise For Debugging – Disabled 16
4.5 Port Selection 17
5. Downloading PlainFlightController Software 17
6. How To Program 19
7. How To Configure Your Model (Config.hpp) 20
7.1 Section 1: Board & Hardware Selection 20
7.2 Section 2: Receiver 20
7.3 Section 3: Model Type 21
7.3.1 PLANE_FULL_HOUSE 22
7.3.2 PLANE_FULL_HOUSE_V_TAIL 23
7.3.3 PLANE_RUDDER_ELEVATOR 24
7.3.4 PLANE_ADVANCED_RUDDER_ELEVATOR 24
7.3.5 PLANE_V_TAIL 25
7.3.6 PLANE_FLYING_WING 26
7.3.7 Multicopters 26
7.4 Section 4: Output Assignment 31
7.4.1 Model Outputs 31
7.4.2 Servo/Motor/Pass Through Refresh Rate 31
7.4.3 SERVO_REFRESH_RATE 32
7.5 Section 5: RC Function Assignment 33
7.6 Section 6: Output Configuration 33
7.6.1 REVERSE_SERVO_x 33
7.6.2 EXTEND_SERVO_TRAVEL_RANGE 33
7.6.3 CALIBRATE_ESC 33
7.7 Section 7: Feature Flags 34
7.7.1 USE_FLAPS (Fixed Wing Only) 34
7.7.2 USE_DIFFERENTIAL_THRUST (Fixed Wing Only) 34
7.7.3 USE_HEADING_HOLD (Fixed Wing Only) 34
7.7.4 USE_LOW _VOLTS_CUT_OFF 34

7.7.5 USE_EXTERNAL_LED 34
7.7.6 USE_ACRO_TRAINER 35
7.7.7 HAS_TELEMETRY 35
7.7.8 USE_PROP_HANG_MODE (Fixed Wing Only) 35
7.8 Section 8: Optional GPS 35
7.8.1 GNSS_TYPE 35
7.8.2 GENERATION 35
7.9 Section 9: Flight Characteristics 36
7.9.1 GYRO_RATE 36
7.10 Section 9: Multicopter Motor Settings 37
8. Transmitter Configuration 38
8.1 Trims 39
8.2 Arming Switch 39
8.3 Mode Switch 39
8.3.1 Pass Through Mode 39
8.3.2 Rate Mode 39
8.3.3 Self-Levelled Mode 39
8.3.4 Acro Trainer Mode 39
8.4 Flaperons Switch (Now channel selectable) 39
8.5 Heading Hold Switch 40
8.6 Prop-hanging Mode Switch 40
9. How To Test 40
9.1.1 LED Status 40
9.2 DEBUG 41
9.2.1 DEBUG_GYRO_CALIBRATION 42
9.2.2 DEBUG_MPU6050 42
9.2.3 DEBUG_MADGWICK 42
9.2.4 DEBUG_LOOP_RATE 43
9.2.5 DEBUG_RX 43
9.2.6 DEBUG_RC_DATA 43
9.2.7 DEBUG_BATTERY_MONITOR 44
9.2.8 DEBUG_OUTPUT 44
9.2.9 DEBUG_GPS 45
10. How To Install In Your Model 45
11. How To Tune 45
11.1 WiFi Web Configurator 46
11.1.1 How To Connect 46
11.1.2 PIDF Configuration 48
11.2 Reprogramming ESP32-S3 To Tune Parameters 52
11.3 Rate Mode Tuning 52
11.4 Self-Levelled Mode Tuning 53
11.5 Heading Hold (Fixed Wing Only) 53
11.6 Model Gain Examples 53
11.6.1 FLYING_WING with USE_DIFFERENTIAL_THROTTLE 54
11.6.2 PLANE_FULL_HOUSE 54
11.6.3 PLANE_FULL_HOUSE 55
11.6.4 PLANE_FULL_HOUSE 56
11.6.5 PLANE_FULL_HOUSE 57
11.6.6 PLANE_ADVANCED_RUDDER_ELEVATOR 58
11.6.7 QUAD_X_COPTER 60
12. Pre-Flight Checks 60
13. Disclaimer & License 61

1. Introduction

1.1 Background

PlainFlightController stabilisation software is for the RC pilot who wants to get the most from their model, needs to master an unstable aircraft, or simply counteract environmental conditions for an enjoyable flight.

Originally created as a home project it quickly became more with its performance, ease of build and low budget parts. These qualities led to it being refined and posted on GitHub for other hobbyists to have a go and enjoy.

While PlainFlightController has been developed for small electric powered aircraft it could easily be modified and used on other small radio-controlled craft. This can be done with little effort as the code is broken down into logical classes and is well commented for those that want to understand or modify for their own purposes.

The original flight controller hardware is based on the Seeed Studio XIAO ESP32 boards and the ever-popular MPU6050 IMU. However, more recently the Waveshare Zero and Tiny boards have been adopted. It's simple to build, easy to program via Arduino IDE and cheap when compared to many commercially available flight controllers.

1.2 PlainFlightController V3.x.dev Development Release

V3.x.dev brings many improvements and upgrades to the V1 code base. The main points to note are as follows:

1.2.1 Channel Pass Through

Any unused PWM outputs can now be assigned to directly output transmitter channels. These can be used for undercarriage and lights etc.

1.2.2 CRSF Rx Protocol

Cyberslug has created a new receiver base class and added Crossfire 250Hz receiver protocol. Additionally a telemetry manager has been added to downlink battery and GPS data.

1.2.3 IBus Rx Protocol

IBus has been added, the implementation does not include telemetry presently.

1.2.4 SRXL2 Rx Protocol

Cyberslug has also added SRXL2 for Spektrum users, the implementation does not include telemetry presently.

1.2.5 GPS

Cyberslug has created the dualGNSS library that allows CASIC and UBLOX modules to be used. GPS data can then be sent as telemetry to your transmitter. Ground speed, height, coordinate etc are all available.

1.2.6 Web Configurator

Update for the web configurator by Bleutzinn to allow correct operation on iPhone. Also includes a general update to the GUI to improve useability.
Additional updates by PlainFlight have since followed to use WebServer library to create a captive portal to assist apple devices and automate launching of a web browser. Also includes realtime updates of pitch, roll, yaw and battery voltages to assist with calibration.

1.2.7 Board Type Selection

To avoid having to change IO and LED settings in config.hpp for different boards, you can now choose from 4 different EPS32-S3 boards. These being the XIAO, Waveshare Zero, Waveshare Tiny and WSMC. These allows correct mapping of IO's and LED type i.e. neopixel. Alternatively, it allows further board types to be easily added.

1.2.8 Board Orientation

The IMU can now be orientate in any of the 24 orientations possible. Configuration is done by stating which direction the positive axis of the X and Y accelerometer are pointing.

1.2.9 Extended Servo Travel

Some budget servos horns do not align perpendicular to the servo body due to poor spline alignment on the servo. To overcome this, trims can be used on the Web interface to centre the required horns. Previously this would limit the servo travel in the direction the servo has been trimmed. Now with the use of EXTEND_SERVO_TRAVEL_RANGE you can increase total servo travel from 1-2ms to 0.8-2.2ms. This is particularly useful where you have 2 aileron servos on a plane or flying wing. Note your servos must be capable of accepting 0.8-2.2ms input for this to work.

1.2.10 SoftWire Library

A move to SoftWire library for I2C communications has been implemented as this offers a ~250us speed up on every loop iteration over the existing Arduino Wire library.

1.2.11 Config Layout

Config.hpp has been reformatted to make it easier for users to understand and find the configuration settings they require.

1.2.12 General Updates

Many small improvements have also been made to code structure. These have helped simplify the code and improve efficiency.

1.3 Specifications

PlainFlightController has the following specifications as detailed in Table 1 as standard:

Table 1 - Specifications

Feature	Detail
Model Mixes	Plane Full House (aileron/flaps/elevator/rudder) Plane Full House V-Tail (aileron/flaps/V-tail mix) Plane V-Tail (V-tail mix for rudder/elevator) Plane Rudder/Elevator (rudder/elevator) Plane Advanced Rudder/Elevator (rudder/elevator) Flying Wing (elevons/rudder) Quad X Quad + Bicopter Chinook Singlecopter Dualcopter Tricopter
Flight Modes	Pass Through (Fixed wing only) Gyro Rate Self-Levelled Acro Trainer Heading Hold (Fixed wing only) Prop Hang Self-Levelled (Fixed wing only, Plane or tail sitter modes)
Actuators	Maximum of 8 PWM outputs in any combination of servos or motors.
Actuator Refresh Rates	50Hz, 100Hz, 150Hz, 200Hz, 250Hz, 300Hz, 350Hz Oneshot125 (limited to 2KHz)
Motors	Fixed Wing: Standard throttle response, single or twin motors. Or, throttle/PIDF yaw based differential thrust can be configured for twin motors for any fixed wing model mixer.
Radio Protocols	SBus, CRSF (includes telemetry), IBus, SRXL2
Battery Monitor	1-3s Lipo as standard, actively reduces power to motor when voltage drops below a defined threshold.
Failsafe	Fixed Wing: Automatic transition to self-levelled mode and cuts throttle. Multicopter: Cuts motors.
LED	Flight mode indication. Standard LED or Neopixel.
Target	ESP32S3 - Seeed Studio XIAO, Waveshare Zero, Waveshare Tiny, or, any other ESP32S3 with new board configs added.
IMU	MPU6050 (GY-521 breakout board)
Control Loop	Stable 1KHz

Additional Features	Multicopters have an interpretation of BetaFlight's Air Mode and Anti-Gravity to maintain full PID control at minimum & maximum throttle settings.
WiFi Web Configurator	Configure gains and trims with you smart device or PC.
GPS	CASIC and UBlox M6/7/8/9/10 for telemetry purposes only when used with CRSF.
Channel Pass Through	Direct Tx channels to spare PWM channels.

1.4 Scope

PlainFlightController software is only intended to be used on small electrically powered model aircraft, with possible adaptions for other small radio-controlled craft i.e. Hovercrafts, hydrofoils, cars etc.

No warranty or guarantee is given with this software or accompanying documentation, refer to license agreement for full details on using this software.

2. How To Assemble

It is recommended that the flight controller is built onto PCB prototype board for rigidity and robustness, but for the lightest weight application it can be built with 'flying wires' connecting between the components.
Figure 1 and Figure 2 detail the wiring diagrams of both 5V (BEC powered) and 3.3V (1s Lipo powered) systems. They both show connections for a 'full house' plane with 4 servo outputs and 2 motors. This wiring configuration can be used for any of the 'Model Mixers' in the software as all servo/motor outputs are given.

Note: Servo/motor power supplies have been omitted for clarity.
Note: The ESP32S3 I/O's are not stated as 5V tolerant and needs the series resistors to drop voltage / current limit when interfacing 5V logic. If you know your servos/ESC uses 3V3 logic then these can be omitted.
Note: 470uF 16V capacitor protects the ESP32S3 from voltage spikes. Recommended when the ESP32 is powered directly from a single lipo with a brushless motor in the system.

Figure 1 – 5V Wiring Diagram

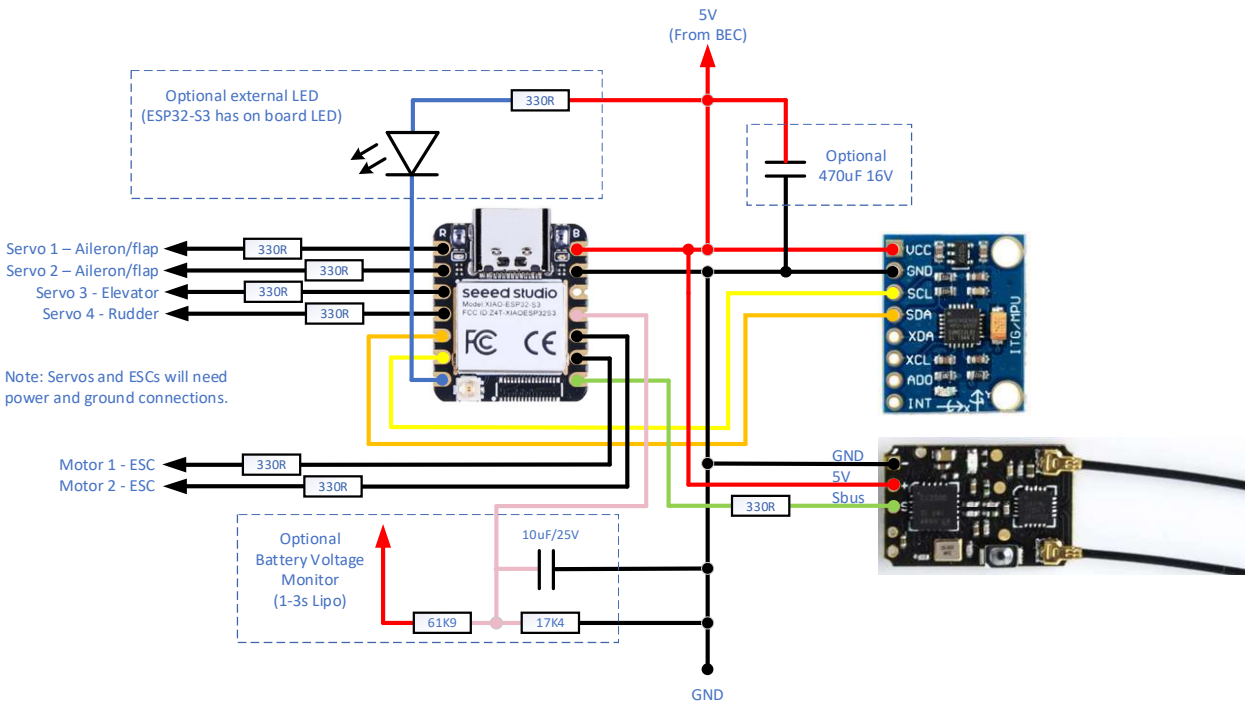
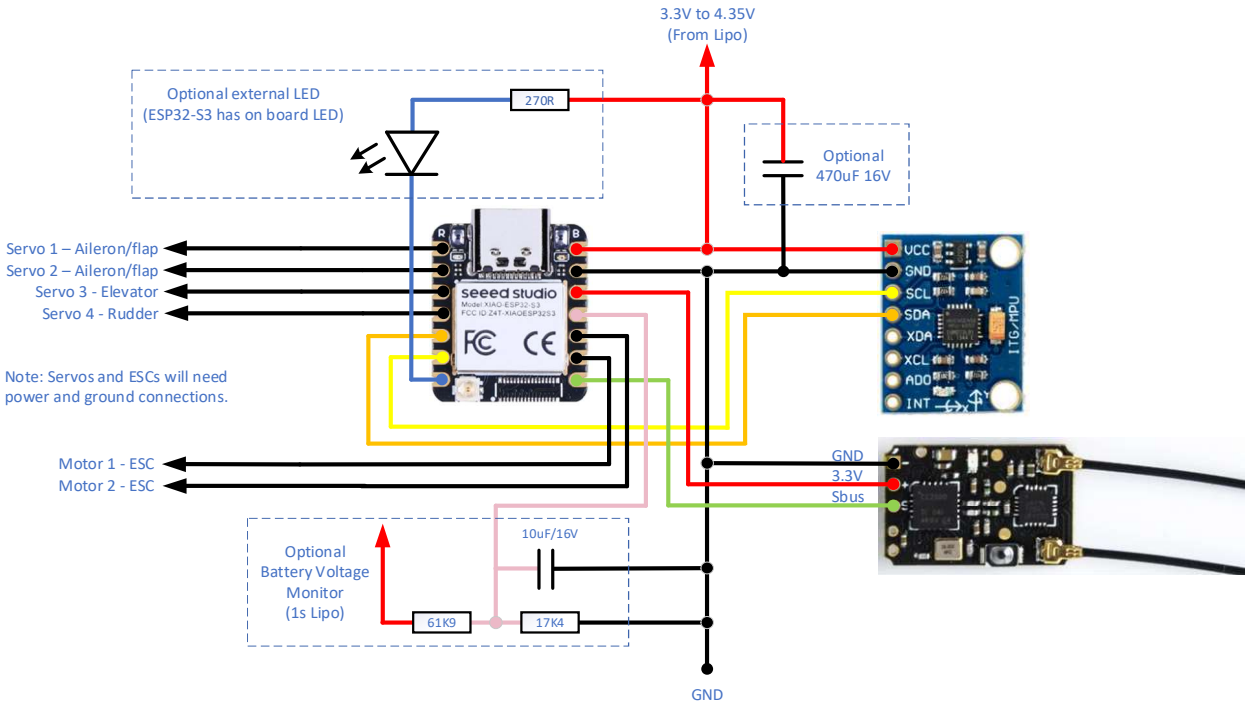


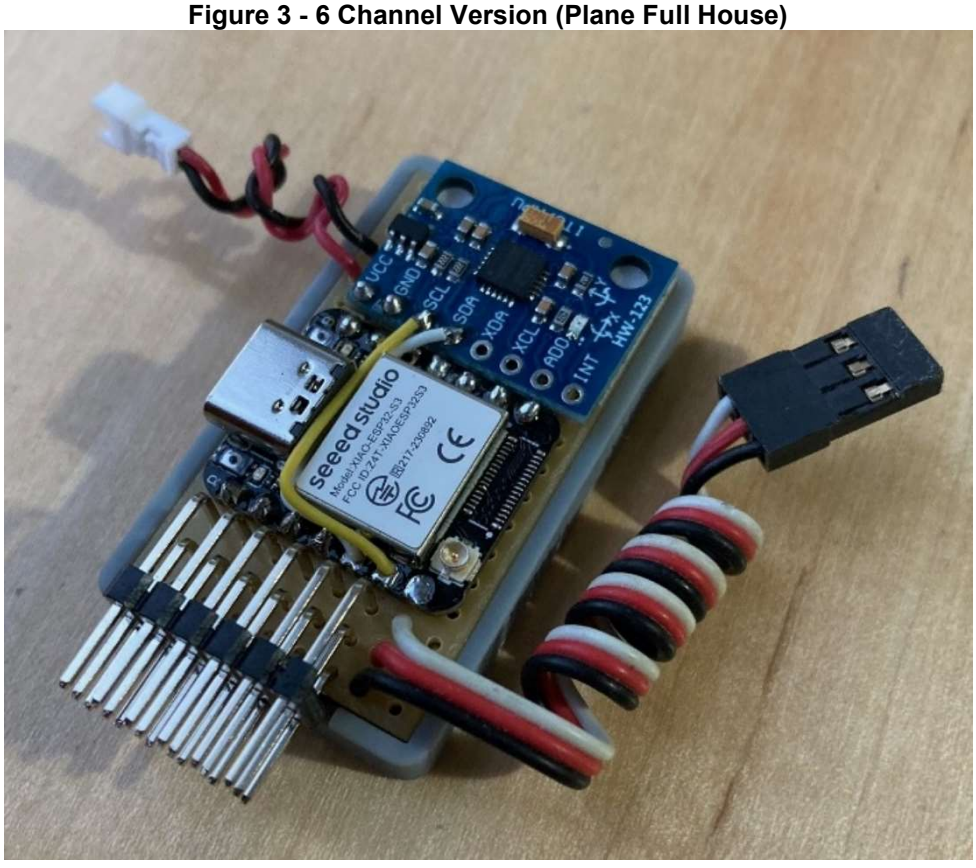
Figure 2 - 3.3V Wiring Diagram



Note: Strongly recommend 470uF 16V capacitor on 3.3V systems supply rail if using BLHeli ESC's to absorb voltage spikes, and generally recommended to prevent the possibility of brown out on 3.3V and 5V systems.

Figure 3 details a typical build on prototype board. This is the 5V version and all resistors are 0805 package soldered onto the back of the prototype board. Note this version does not use the external LED, but does use battery monitoring input.

Figure 3 - 6 Channel Version (Plane Full House)



2.1 Components

To build the flight controller you will need the items listed in Table 2. These can be obtained from a good hobbyist electronics store, eBay, or Amazon.

Table 2 – 5V Flight Controller Components

Component	Quantity	Comment
Seeed Studio XIAO ESP32S3, Waveshare ESP32S3 Zero, Waveshare ESP32S3 Tiny.	1	XIAO ESP32C3 is not directly supported but can be used if you modify the software. Its clock speed is lower and may affect control loop times and you will also be limited to 6 actuator outputs.
GY-521 (MPU6050 breakout)	1	
330 Ohm resistor	8	For servo/motor and receiver; These are only required when interfacing 5V logic components. **Note:** The ESP32-S3 is stated as being not 5V tolerant. However, most modern Rx's, servos and ESC's use 3.3V logic devices - It is your responsibility to check compatibility. The external LED option always requires a series resistor.
10uF 16V/25V capacitor	1	Used to filter/smooth battery voltage and helps remove any electrical voltage spikes in its sampling. If not fitted increase weighted filter in the software.
LED	1	Optional LED for external use. Can use LED_BUILTIN of Arduino.
Sbus/IBus/CRSF Receiver	1	Standard 100Kb Sbus, 250Hz CRSF or IBus receiver.
30AWG Silicone Wire	-	Multiple colours for wiring of components.
61K9 0.25W	1	If not using exact values software can be modified. Ensure potential divider output does not exceed 3.3V.
17K4 0.25W	1	If not using exact values software can be modified. Ensure potential divider output does not exceed 3.3V.
PCB Prototype Pad Board (2.54mm/0.1 inch pitch)	~42x22mm	Use what you are comfortable with or have. Or build the controller with 'flying wires'.
1mm double sided foam tape	20mm	Used to soft mount gyro.

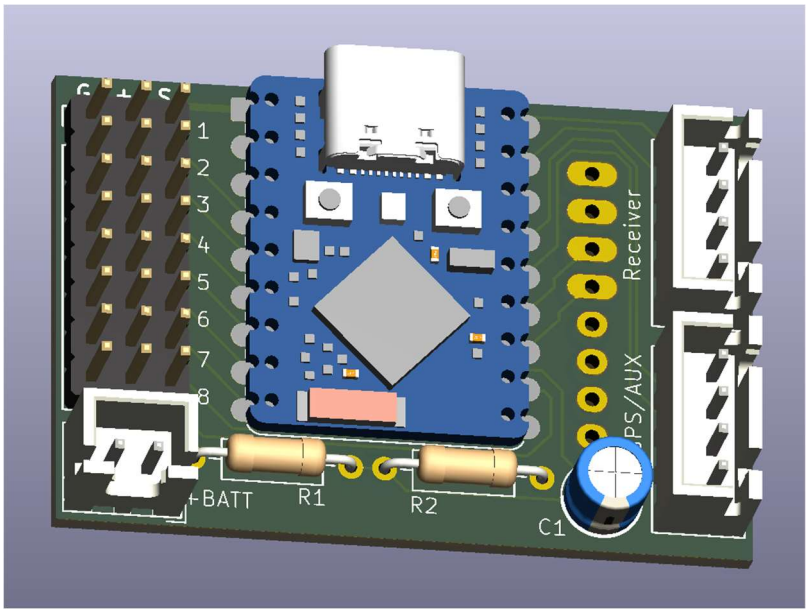
3. PCB's

This project offers PCB's that you can order from pcbway.com or jlcpcb.com. Just copy and paste the zipped files in the manufacture folder onto their websites to get an instant quote. Ordinarily 5-10 bare PCBs with 8-10 day postage comes in under $15US.

PCB files can be found in the PCB folder on the GitHub project page.

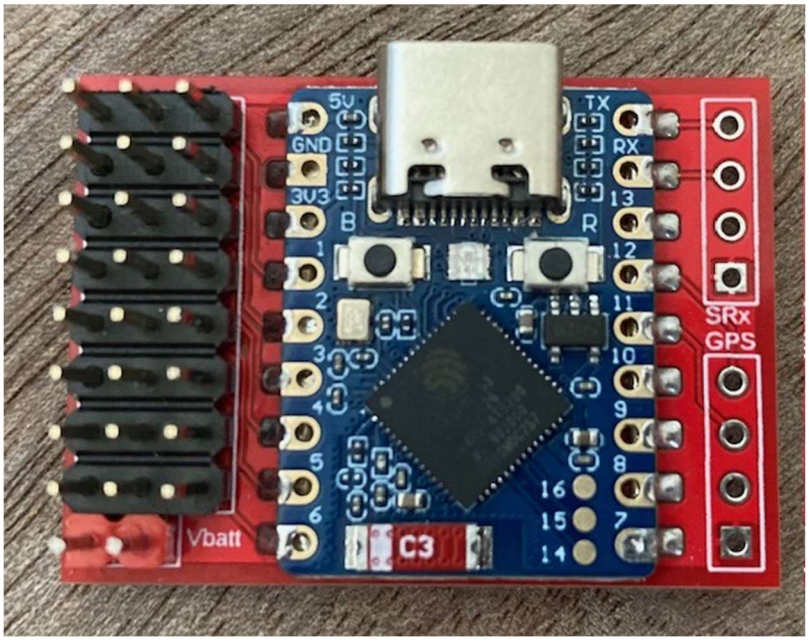
3.1 WSMC

The Waveshare module carrier (WSMC) is a through hole design that is easy to build. It supports all software features provided by Plain Flight Controller.



3.2 WSMC Mini

The Waveshare module carrier Mini (WSMC Mini) is a SMT design. Suitable for smaller models but does require a few discrete 0805 components to be soldered. As per WSMC it supports all software features provided by Plain Flight Controller.



4. Arduino IDE Installation

For this installation process it is assumed you will be using the Seeed Studio XIAO ESP32-S3. The installation process should be the same for other ESP32 Arduino boards. This document examples are based on Arduino version 2.3. Other versions may vary in layout/appearance.

4.1 Arduino IDE Installation

For your operating system download and install the newest version of Arduino IDE.

https://www.arduino.cc/en/software



Once installed launch the Arduino application.

4.2 Arduino Library Installation

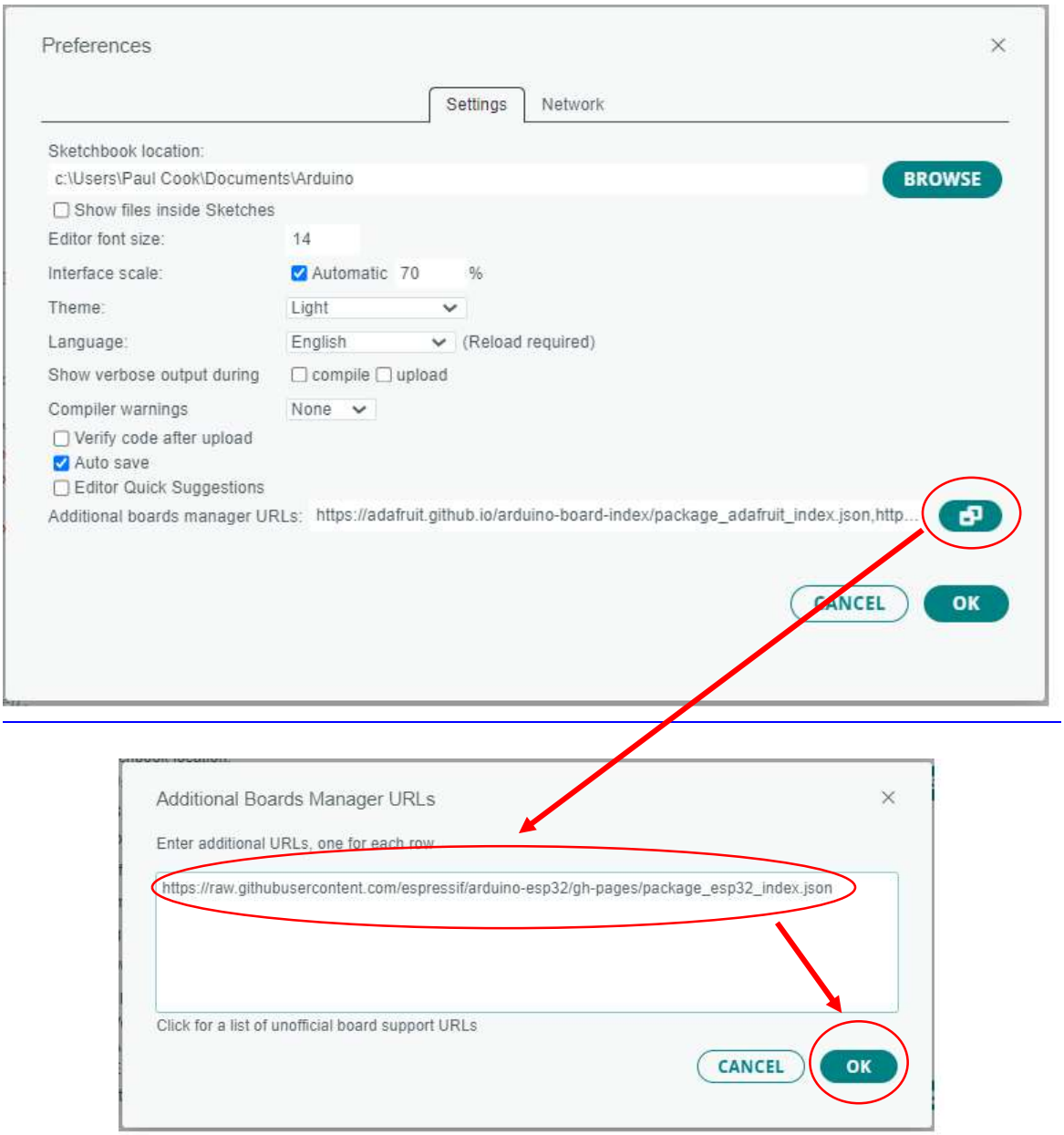
To compile the ESP32 boards for Plain Flight Controller you will need to install libraries into your Arduino IDE.

4.2.1 Espressif Systems ESP32 Installation

First you need to map the 3rd party library location . To do this copy the URL given below then navigate to **File > Preferences**, and in the pop-up window fill **'Additional Boards Manager URLs'** with the following URL as shown by Figure 4:

https://espressif.github.io/arduino-esp32/package_esp32_index.json

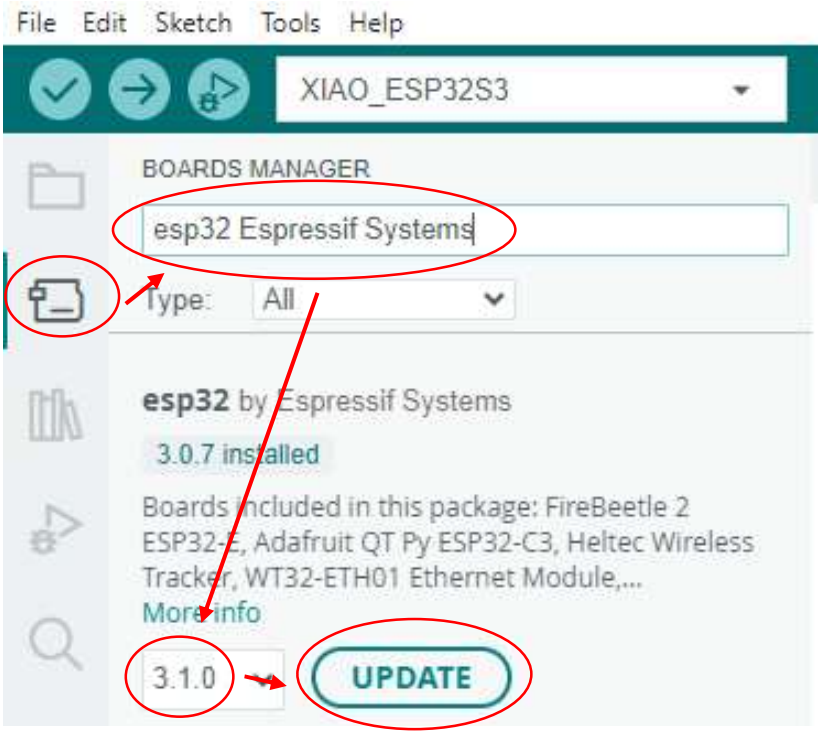
Figure 4 - Espressif Systems URL Mapping



Note: Click **'OK'** to accept and close all pop-up windows.

Now to actually install the 'Espressif Systems ESP32' board package into the Arduino IDE; Navigate to **Boards Manager** icon, type the keywords '**esp32 Espressif Systems**' in the search box, select version 3.1.0 and click the Update/Install button as indicated by Figure 5.

Figure 5 - ESP32 Espressif Systems Board Package 3.1.0

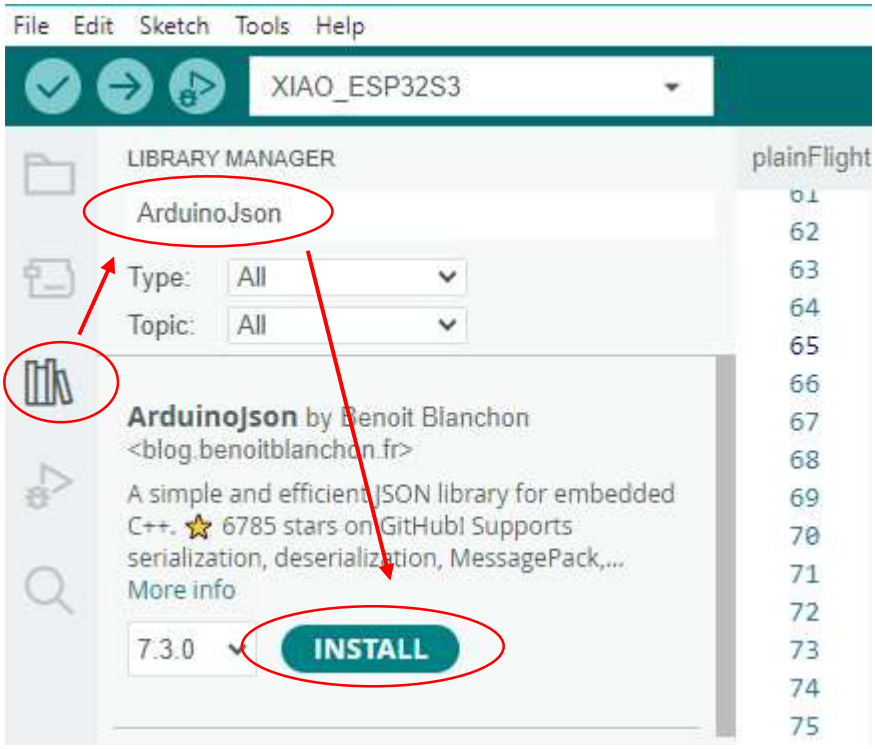


Should you have any problems installing this please refer to:

https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html

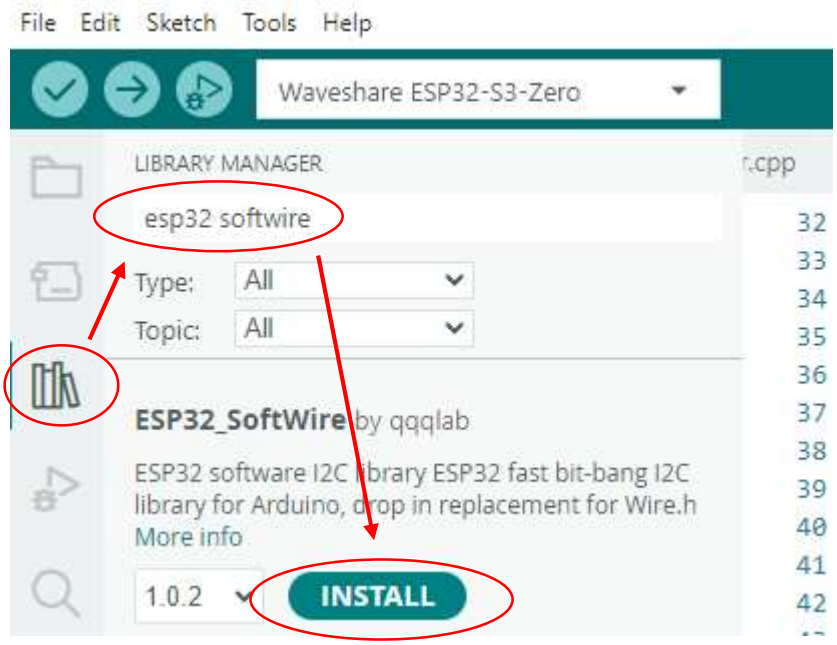
4.2.2 ArduinoJson

Install the ArduinoJson library as this is needed for the file system. Library version 7.3.0 has been used and tested with PlainFlightController V2.0.0.



4.2.3 ESP32_SoftWire

Install the ESP32_SoftWire library, version 1.0.2 has been used and tested with PlainFlightController.



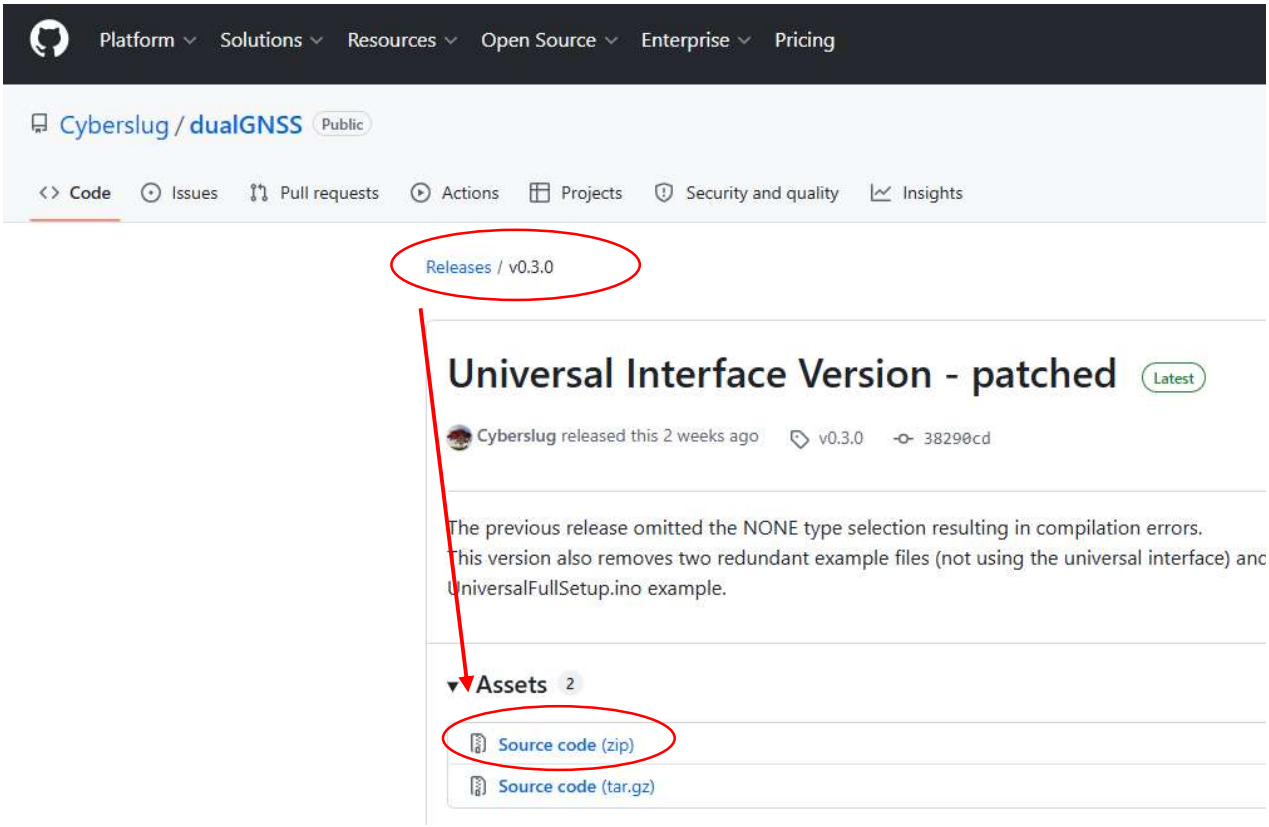
4.2.4 dualGNSS GPS Library

This is a binary GNSS library for CASIC and UBX protocol modules. The dualGNSS library produces a comprehensive navigation data packet with speed and position data. This data can then be sent via CRSF telemetry protocol to your transmitter for display.

Obtain the dualGNSS library by Cyberslug from https://github.com/Cyberslug/dualGNSS/

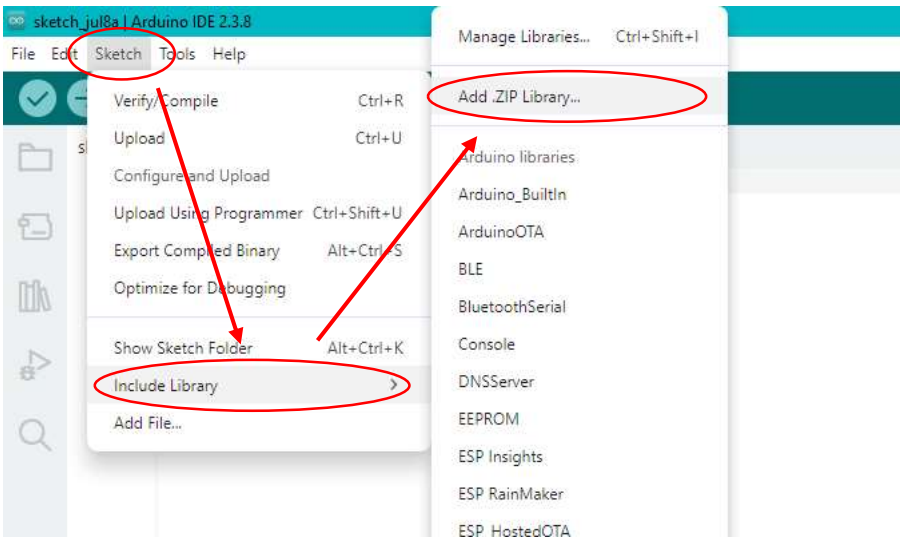
Note: The code base will not build without this library even if you are not using a GPS.

Go to the releases and download the latest zip source release:



Place the downloaded library zip folder into C:\Arduino\libraries or the equivalent path for where your Arduino project folder is located.

In Arduino navigate to ‘Sketch->Include Library->Add .Zip Library...’ then select the dualGNSS library you just downloaded.

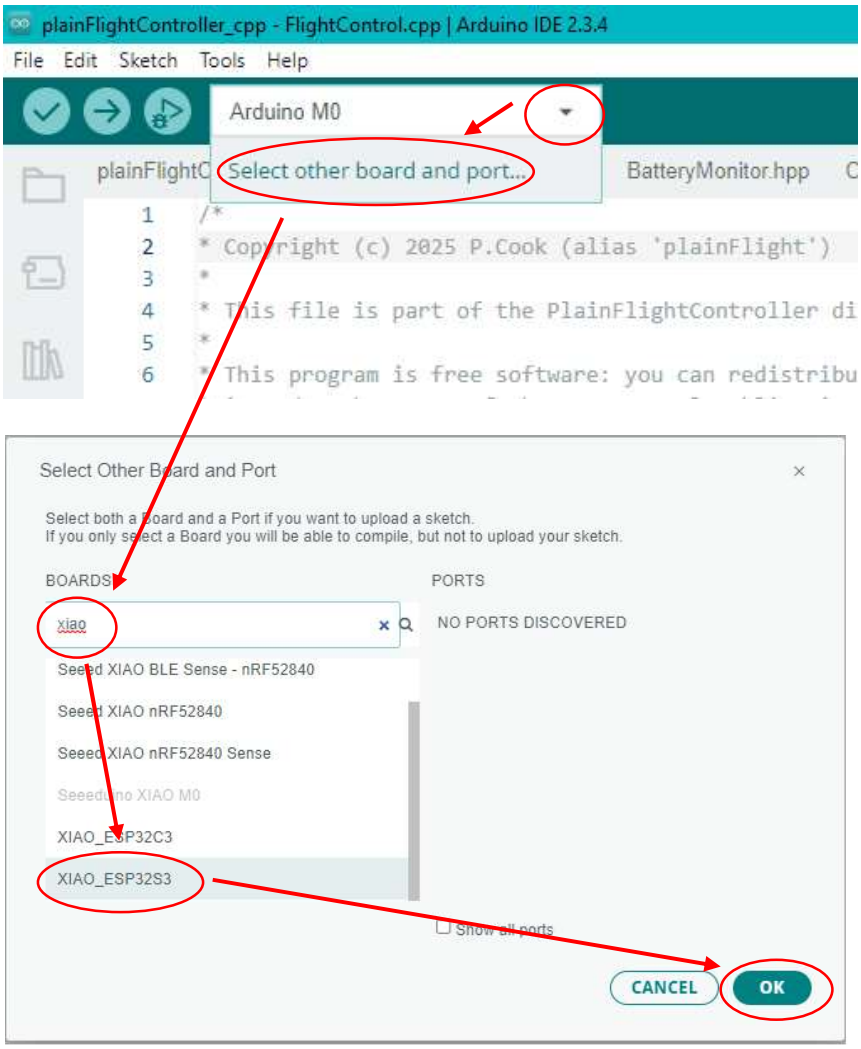


4.3 ESP32-S3 Board Selection

To compile and program the XIAO ESP32S3 you need to select the correct board. Figure 6 details how to navigate to drop down board menu and **‘select other board and port...’**, from the pop up window find and

select **XIAO_ESP32S3**, **Waveshare ESP32-S3-Zero**, **Waveshare ESP32-S3-Tiny**, or the ESP32-S3 board you are using from the list of boards.

Figure 6 - XIAO ESP32S3 Board Selection

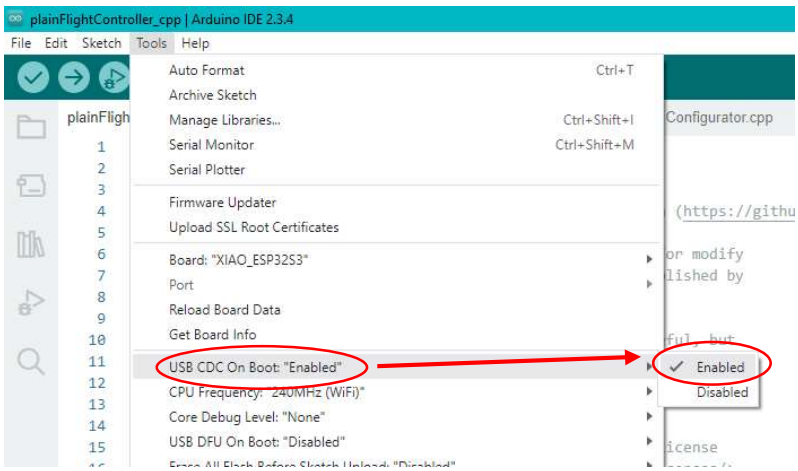


4.4 IDE ESP32 Tools Configuration

4.4.1 USB CDC On Boot

For serial communications to work you must ensure that USB CDC OnBoot is enabled. This is found via Tools->USB CDC On Boot->Enabled.

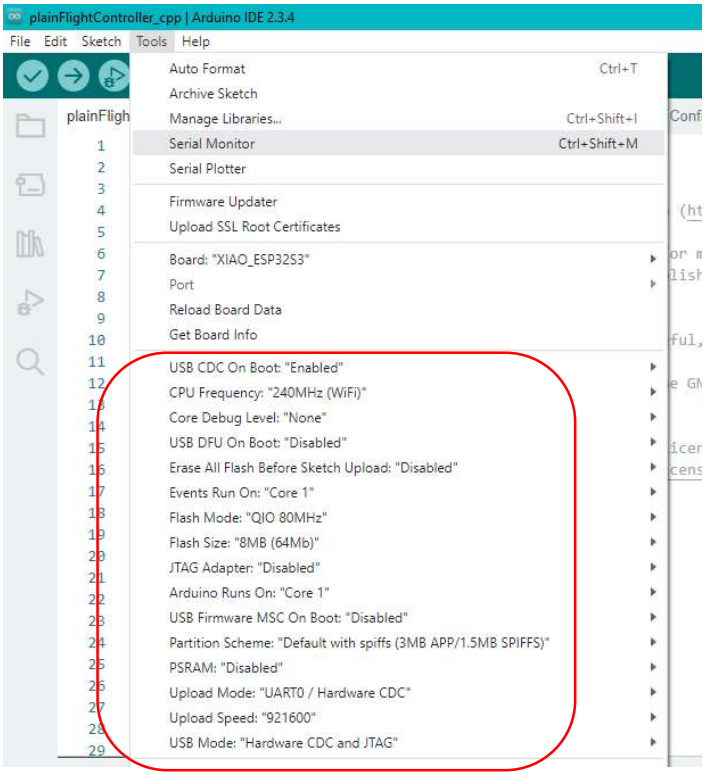
Figure 7 - Enable USB CDC On Boot



4.4.2 Other Tool Configurations

The other setting should default but ensure all other settings are as follows, particularly the Flash Mode of QIO:

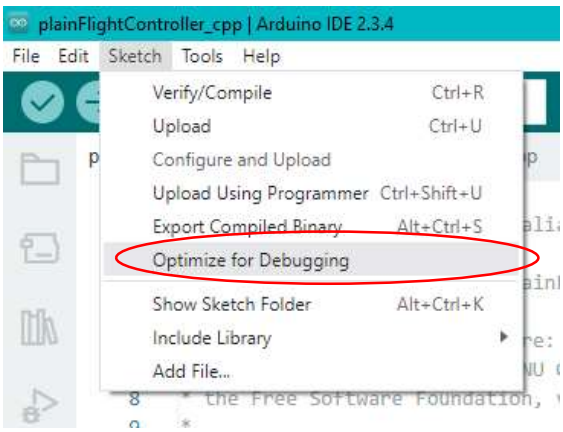
Figure 8 - Default Settings



4.4.3 Optimise For Debugging – Disabled

You must ensure that 'Optimise For Debugging' is **disabled**. If it is enabled, you will not consistently achieve a stable 1Khz loop rate.

Figure 9 - Disable Optimise For Debugging



4.5 Port Selection

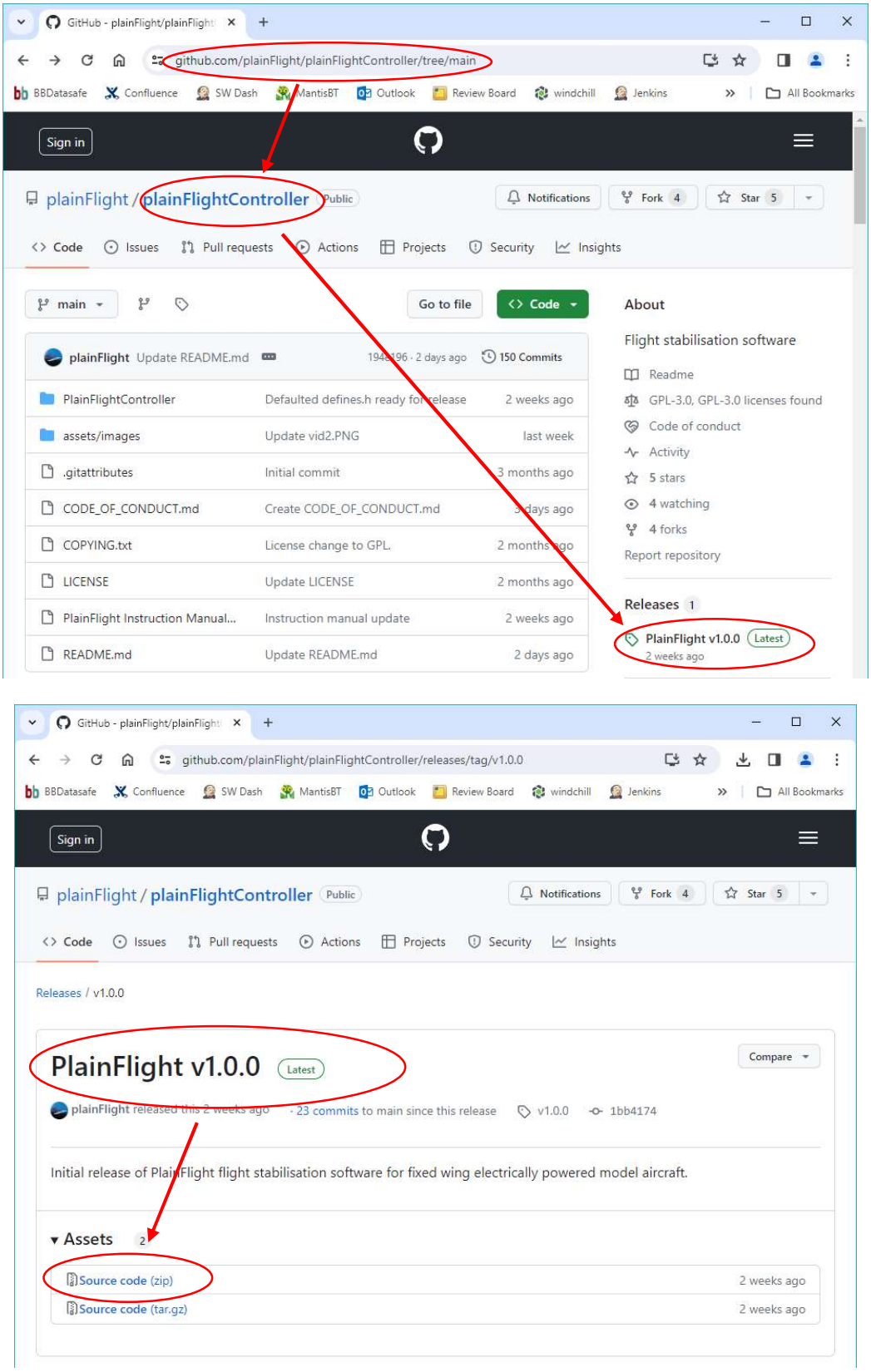
Connect the XIAO_ESP32S3 to your computer via USB. Wait for the drivers to be automatically installed then navigate to **Tools > Port** and select the serial port name of the connected ESP32S3.

Note: The ESP32S3 boards often comes up with the wrong name, so select the named board that appears on USB connection.

5. Downloading PlainFlightController Software

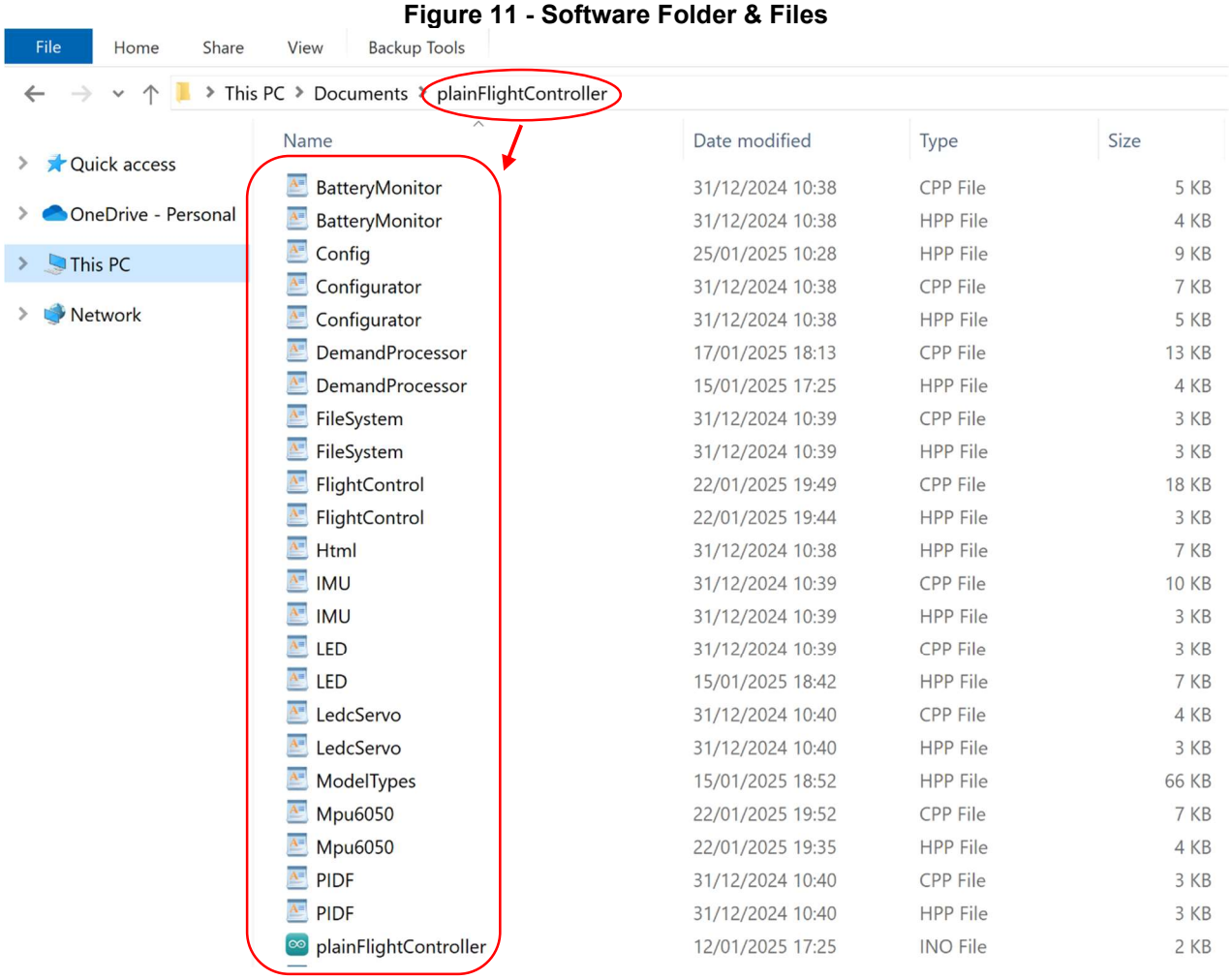
From github.com search ‘plainFlightController’ to find the project home page. To download the flight controller software, click on the latest release version (original v1.0.0 shown in example below) then select ‘Download ZIP’. This will download the Arduino code, instructions and license to your computer. Figure 10 details the sequence for downloading the software.

Figure 10 - PlainFlightController Software Download



Once downloaded unzip the files at a suitable location. Note the software files should appear inside a folder called 'PlainFlightController'. If they do not create it and place the software files inside it, as detailed by Figure 11, or it will not compile.

Figure 11 - Software Folder & Files



6. How To Program

CAUTION: Always remove the flight battery when programming. The motor may start unexpectedly if you do not.

CAUTION: If you have built the 3.3V system never plug USB and 1s lipo flight battery in at the same time as the 5V USB supply will back feed the 4.2V lipo battery.

Open the Arduino project by double clicking 'PlainFlightController.ino' file. The Arduino IDE should open and all of the software files will appear as tabs inside the IDE.

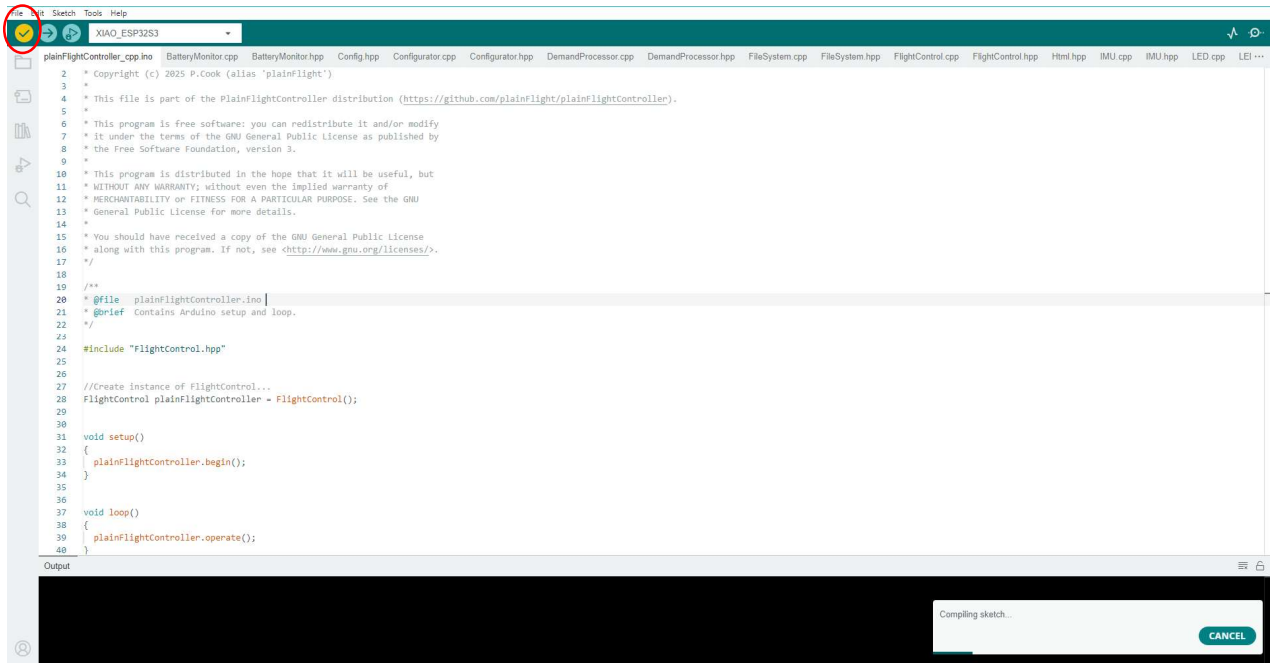
Ensure the correct target board is selected (ESP32S3 family) as detailed in section 4.3.

Plug in your ESP32S3 to your computer via the USB and select the correct port as detailed in section 4.5.

You can now select the upload button to program your ESP32S3 with PlainFlightController software. Figure 12 shows how the Arduino IDE should appear with all software files, correct board selected, and upload button highlighted.

The Upload can take several minutes, especially if it's the first time the software has been compiled, a progress bar will be given to indicate the upload status.

Figure 12 - PlainFlightController Upload



Should your ESP32S3 board fail to program, disconnect power/USB and hold down the boot button on the ESP32S3 board. Now reapply power to enter boot mode ready for programming.

7. How To Configure Your Model (Config.hpp)

The flight controller is configured via the constant expressions contained within Config.hpp file. The file is well commented, but the following sections describe these in detail.

7.1 Section 1: Board & Hardware Selection

Board type is intended to provide a quick configuration for different ESP32S3 boards. Currently 3 boards are supported:

Board	Definition
Seeed Studio XIAO	XIAO
Waveshare Zero	ZERO
Waveshare Tiny	TINY

If you wish to add new board configuration look at BoardConfig.hpp.

7.2 Section 2: Receiver

You have the choice of Sbus (142Hz refresh rate), Crossfire CRSF (250Hz refresh), IBus (115kbits / 142Hz), or Spektrum SRXL2 (115.2kbits) serial protocols. Choose the protocol that is applicable to your radio or needs.

Rx Protocol	Definition
SBus (100kbits / 142Hz)	SBUS
Crossfire (400kbits / 250Hz)	CRSF
IBus (115kbits / 142Hz)	IBUS
SRXL2 (115.2kbits)	SRXL2

Note: SRXL2 requires the Rx pin to directly connect to the Tx pin through a Schottky diode (cathode towards the Tx). You may also require a pull up resistor if the Rx does not do so.

7.3 Section 3: Model Type

There is currently a choice of 13 model types to choose from, 6 of which are fixed wing configurations and 7 are multicopter configurations. For the fixed wing configurations differential thrust can be set (if using 2 motors) and self-levelled prop-hang modes can be set.

The model type is set via the constant expressions within Config.hpp. Choose the model type by assigning your choice from the following:

Figure 13 - Model Type Setting

Model Type	Definition
PLANE_FULL_HOUSE	2 aileron servos, flaperons, elevator, rudder, up to 2 motors
PLANE_FULL_HOUSE_V_TAIL	2 aileron servos, flaperons, elevator/rudder V-tail mix, up to 2 motors
PLANE_ADVANCED_RUDDER_ELEVATOR	Rudder, elevator, up to 2 motors
PLANE_RUDDER_ELEVATOR	Rudder, elevator, up to 2 motors
PLANE_V_TAIL	Rudder/elevator V-tail mix, up to 2 motors
PLANE_FLYING_WING	Elevons, rudder, up to 2 motors
QUAD_X_COPTER	4 motors
QUAD_P_COPTER	4 motors
BI_COPTER	2 motors, 2 servos yaw/pitch mix
CHINOOK_COPTER	2 motors, 2 servos yaw/roll mix
TRI_COPTER	2 motors, 1 yaw servos
DUAL_COPTER	2 motors, 2 servos pitch/roll mix
SINGLE_COPTER	2 motors, 4 servos pitch/roll/yaw mix

Note: that multicopter mixers only have rate and self-levelled/acro-trainer modes (Pass through is not possible). They also have my own interpretation/implementation of ‘Air Mode’ and ‘Anti Gravity’ which are always active and do not require any tuning parameters.

7.3.1 PLANE_FULL_HOUSE

This mixer offers 2 servos for aileron/flap functions, rudder and elevator, single or dual motor. flaperons have 3 positions and are controlled via Aux1.

Figure 14 - PLANE_FULL_HOUSE

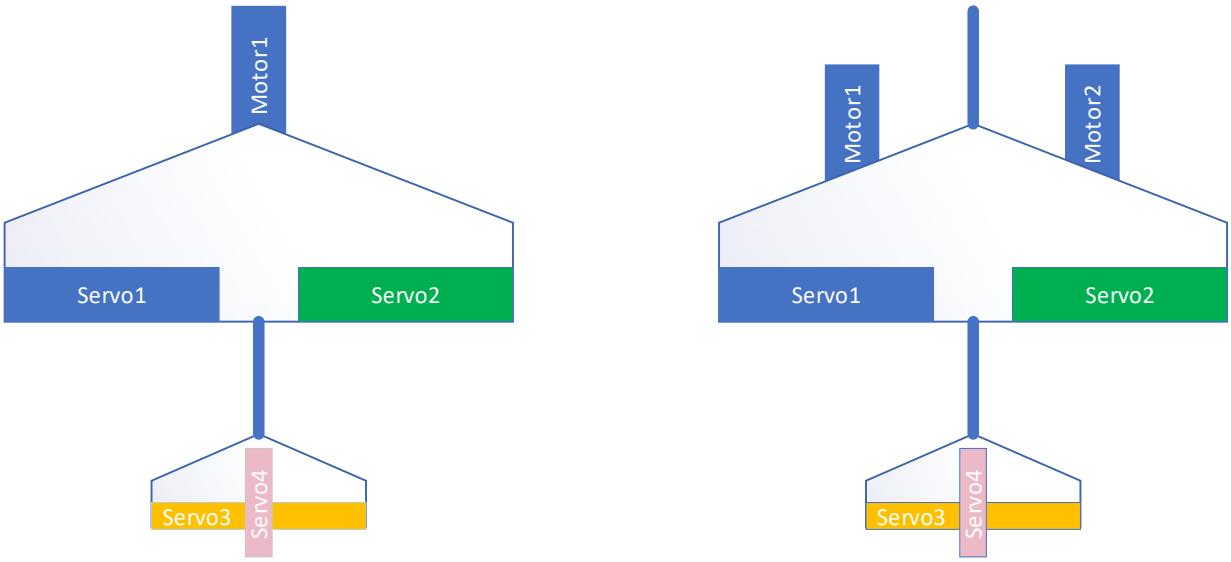


Table 3 - Plane Full House Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle	D8	
Motor 2	Throttle	D9	If fitted on model.
Servo 1	Left aileron/flap	D0	Aux1 operates 3 position flap
Servo 2	Right aileron/flap	D1	Aux1 operates 3 position flap
Servo 3	Elevator	D2	
Servo 4	Rudder	D3	

Note: To enable differential thrust uncomment the define USE_DIFFERENTIAL_THRUST.
Note: To enable Oneshot125 for BLHeli type ESCs set motor RefreshRate to IS_ONESHOT125.
Note: Pusher configuration has no impact on the flight controller mixer.

7.3.2 PLANE_FULL_HOUSE_V_TAIL

This mixer offers 2 servos for aileron/flap functions, and mixed rudder/elevator for a V-tail model. Single or dual motors can be used and flaperons have 3 positions and are controlled via Aux1.

Figure 15 - PLANE_FULL_HOUSE_V_TAIL

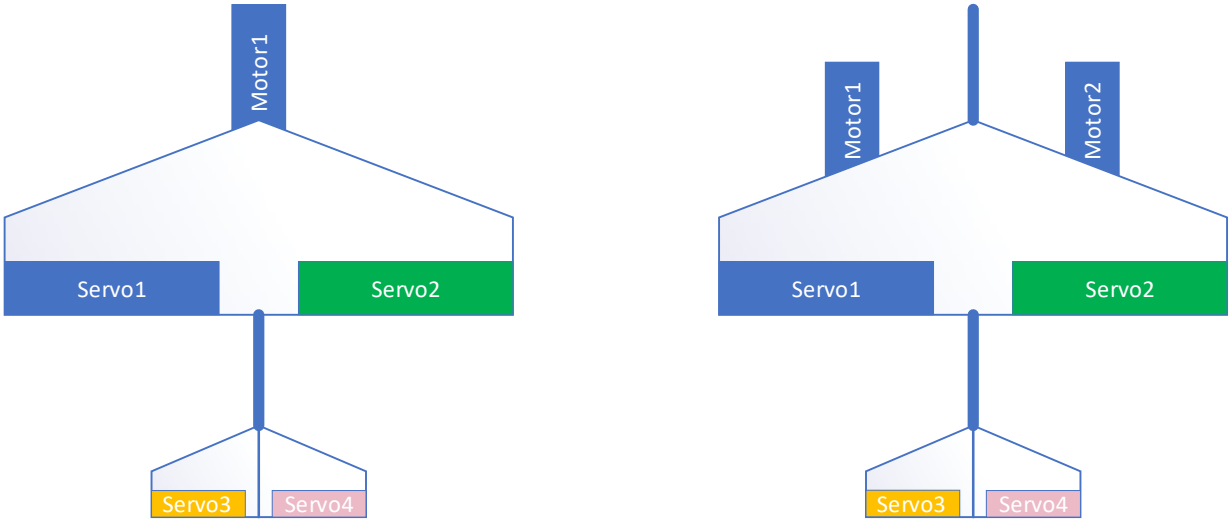


Table 4 - Plane V-Tail Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle	D8	
Motor 2	Throttle	D9	If fitted on model.
Servo 1	Left aileron/flap	D0	Aux1 operates 3 position flap
Servo 2	Right aileron/flap	D1	Aux1 operates 3 position flap
Servo 3	Elevator/rudder	D2	
Servo 4	Elevator/rudder	D3	

Note: To enable differential thrust uncomment the define USE_DIFFERENTIAL_THRUST.
Note: To enable Oneshot125 for BLHeli type ESCs set motor RefreshRate to IS_ONESHOT125.
Note: Pusher configuration has no impact on the flight controller mixer.

7.3.3 PLANE_RUDDER_ELEVATOR

This mixer offers rudder and elevator control with single or dual motors. Rudder is controlled by the roll axis Tx input and is used to stabilise roll axis when in rate or self-levelled modes.

Figure 16 - PLANE_RUDDER_ELEVATOR

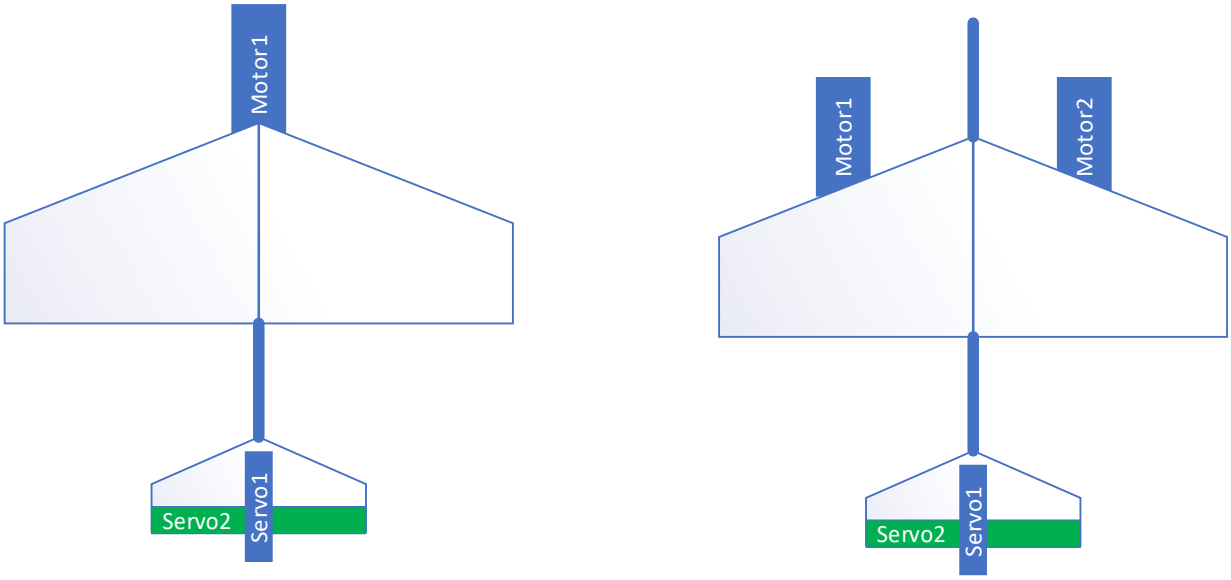


Table 5 - Plane Rudder Elevator Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle	D2	
Motor 2	Throttle	D3	If fitted on model.
Servo 1	Rudder	D0	
Servo 2	Elevator	D1	

Note: To enable differential thrust uncomment the define USE_DIFFERENTIAL_THRUST.
Note: To enable Oneshot125 for BLHeli type ESCs set motor RefreshRate to IS_ONESHOT125.
Note: Pusher configuration has no impact on the flight controller mixer.

7.3.4 PLANE_ADVANCED_RUDDER_ELEVATOR

This mixer offers rudder and elevator control with single or dual motors. Rudder is controlled by the roll axis Tx input and is used to stabilise roll axis when in rate or self-levelled modes. However, rudder is also controlled by the yaw axis Tx input and is used to stabilise yaw axis when in rate or self-levelled modes.

This allows stall turns to be cleanly executed via Tx yaw input and prevents I-gain build up on the roll axis. It also gives a more natural feel to using rudder stick for stall turns and seems to reduce ‘wing rock’ due to the yaw PIDF counter acting roll.

7.3.5 PLANE_V_TAIL

This mixer offers rudder and elevator control with single or dual motors. Use this for rudder/elevator control as flap function is disabled.

Figure 17 - PLANE_V_TAIL

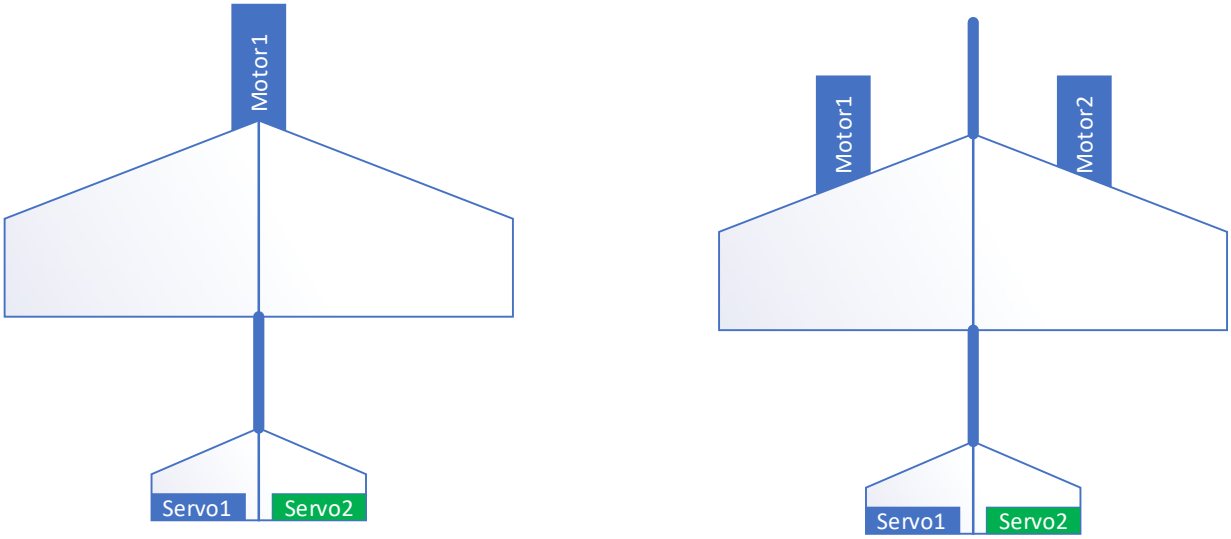


Table 6 - Plane V-Tail Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle	D2	
Motor 2	Throttle	D3	If fitted on model.
Servo 1	Left Rudder/elevator	D0	
Servo 2	Right Rudder/elevator	D1	

Note: To enable differential thrust uncomment the define USE_DIFFERENTIAL_THRUST.
Note: To enable Oneshot125 for BLHeli type ESCs set motor RefreshRate to IS_ONESHOT125.
Note: Pusher configuration has no impact on the flight controller mixer.

7.3.6 PLANE_FLYING_WING

This mixer offers elevon mixing of roll and elevator controls for flying wings. Rudder control is also available if fitted on the model. As standard single or dual motor control is given.

Figure 18 – MIXER_FLYING_WING

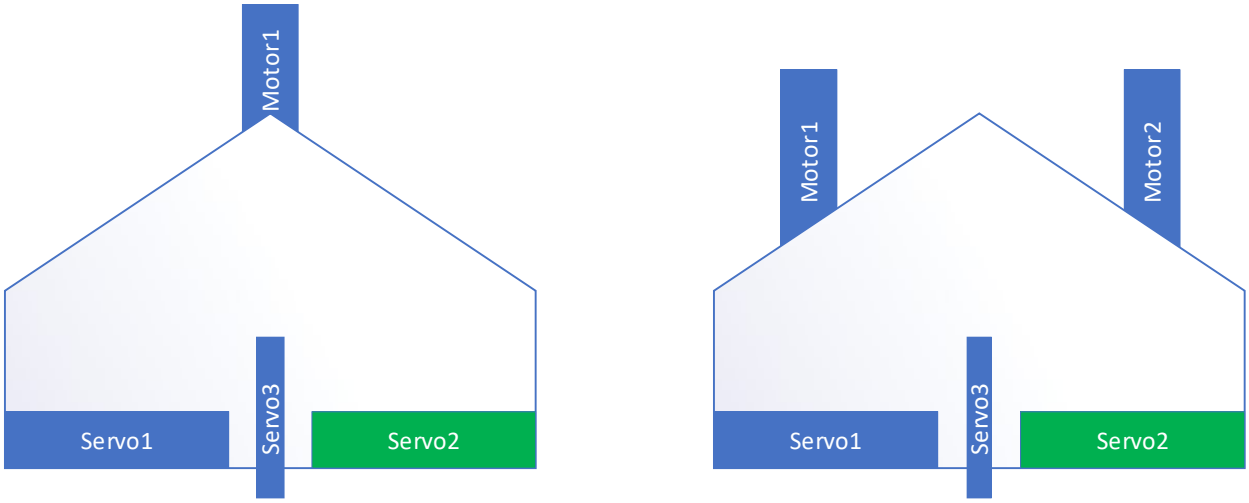


Table 7 - Plane Rudder Elevator Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle	D8	
Motor 2	Throttle	D9	If fitted on model.
Servo 1	Left elevon	D0	
Servo 2	Right elevon	D1	
Servo 3	Rudder	D2	

Note: To enable differential thrust uncomment the define USE_DIFFERENTIAL_THRUST.
Note: To enable Oneshot125 for BLHeli type ESCs set motor RefreshRate to IS_ONESHOT125.
Note: Pusher configuration has no impact on the flight controller mixer.

7.3.7 Multicopters

Do not expect PlainFlightController to fly multicopters with the same level of precision of other open-source flight controllers. However, with correct tuning they can be agile and perform good aerobatics.

Multicopters have selectable flight modes of rate, acro-trainer and self-levelled/acro-trainer. There is also transparent modes similar to BetaFlights ‘Air Mode’ and ‘Anti Gravity’ which are always active. This gives full PID control at throttle extremes and allows you to punch out and free fall under full control with no deviation of the aircraft.

For best performance you are advised to always use motor refresh of IS_ONESHOT125 with Bl-Heli (or similar) ESC’s as this gives the fastest update rates to the ECS’s.

7.3.7.1 Motor Calibration

For multicopters always calibrate your ESC’s before tuning idle up, min motor values and/or the first flight. See calibrate ESC feature in Config.hpp.

7.3.7.2 Motor Audible Control Noise

There is no RC interpolation within PlainFlightController, as a result you will hear ~143Hz noise within the motors when moving the Tx sticks. Do not be alarmed this is just the ESC’s adjusting the speed of the motors to suit the newest RC commands.

7.3.7.3 QUAD_X_COPTER

This mixer offers control of 4 motors arrange as a quad X configuration. In config.hpp set QUAD_X_COPTER to true to enable.

Figure 19 – QUAD_X_COPTER

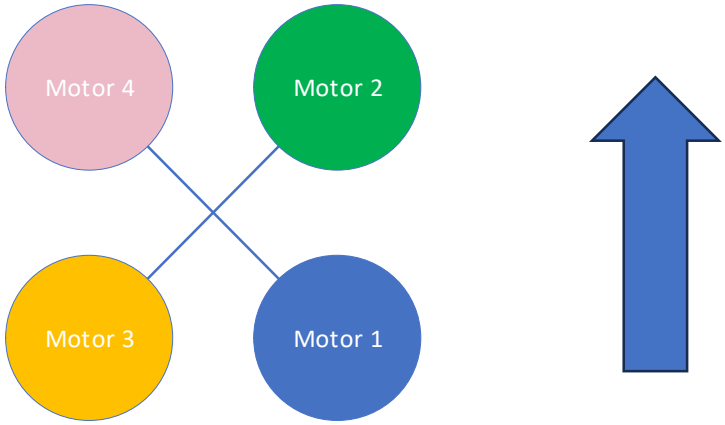


Table 8 – Quad X Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle, roll, pitch, yaw	D0	
Motor 2	Throttle, roll, pitch, yaw	D1	
Motor 3	Throttle, roll, pitch, yaw	D2	
Motor 4	Throttle, roll, pitch, yaw	D3	

7.3.7.4 QUAD_P_COPTER

This mixer offers control of 4 motors arrange as a quad + configuration. In config.hpp set QUAD_P_COPTER to true to enable.

Figure 20 – QUAD_P_COPTER

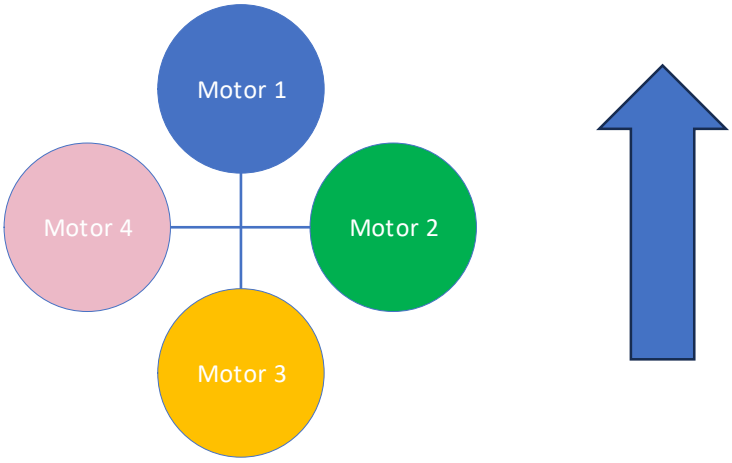


Table 9 – Quad P Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle, roll, pitch, yaw	D0	
Motor 2	Throttle, roll, pitch, yaw	D1	
Motor 3	Throttle, roll, pitch, yaw	D2	
Motor 4	Throttle, roll, pitch, yaw	D3	

7.3.7.5 BI_COPTER

This mixer offers control of 2 motor which are controlled by throttle and roll and 2 servos to mix pitch and yaw controls. In config.hpp set BI_COPTER to true to enable.

Figure 21 – BI_COPTER

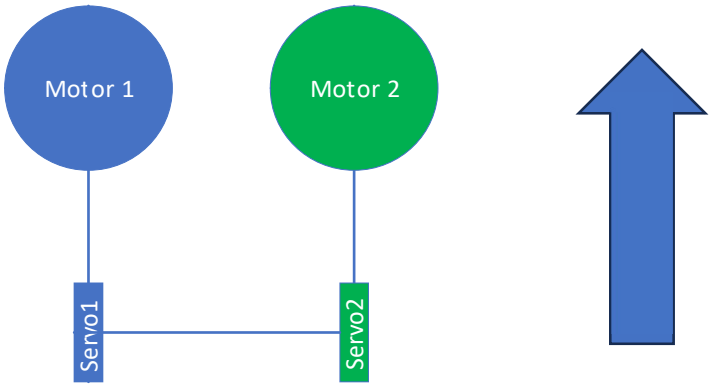


Table 10 – Bicopter Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle, roll	D0	
Motor 2	Throttle, roll	D1	
Servo 1	Pitch, yaw	D2	
Servo 2	Pitch, yaw	D3	

7.3.7.6 CHINOOK_COPTER

This mixer offers control of 2 motor which are controlled by throttle and pitch demands and 2 servo's to mix roll and yaw controls. In config.hpp set CHINOOK_COPTER to true to enable.

Figure 22 – CHINOOK_COPTER

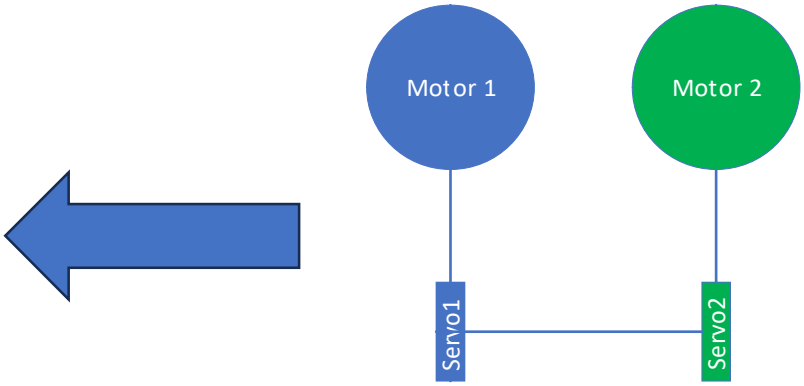


Table 11 – Chinook Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle, pitch	D0	
Motor 2	Throttle, pitch	D1	
Servo 1	Roll, yaw	D2	
Servo 2	Roll, yaw	D3	

7.3.7.7 SINGLE_COPTER

This mixer offers control of 1 motor which is controlled by throttle demand and 4 servos arrange in a + configuration which mix roll, pitch and yaw controls. In config.hpp set SINGLE_COPTER to true to enable.

Note: The yaw servo must have sufficient I-gain to hold heading and overcome motor torque.

Figure 23 – SINGLE_COPTER

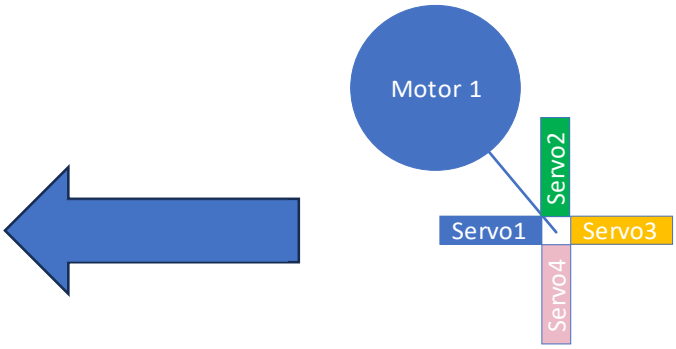


Table 12 - Singlecopter Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle, pitch	D0	
Motor 2	Unused	D1	Must declare as LEDC channel pairs share a common timer which prevents different PWM frequencies i.e. motor refresh and servo refresh.
Servo 1	Roll, yaw	D2	
Servo 2	Pitch, yaw	D3	
Servo 3	Roll, yaw	D8	
Servo 4	Pitch, yaw	D9	

7.3.7.8 DUAL_COPTER

This mixer offers control of 2 motors which are controlled by throttle and yaw demands and 2 servos arrange in a + configuration for Roll and pitch control. Motors must be coaxial design or mounted directly above/below each other. In config.hpp set DUAL_COPTER to true to enable.

Figure 24 – DUAL_COPTER

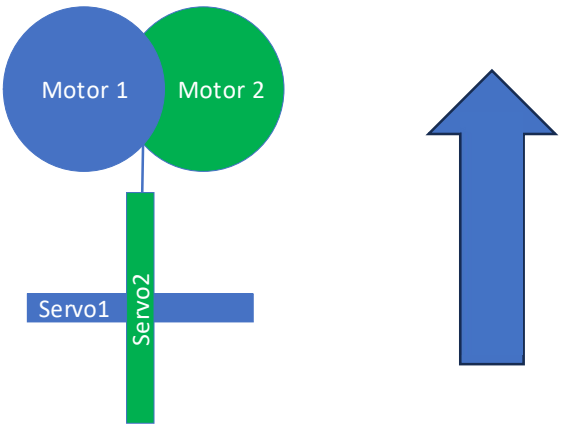


Table 13 – Dualcopter Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle, yaw	D0	Must be coaxial motors, or coaxially mounted.
Motor 2	Throttle, yaw	D1	
Servo 1	Pitch	D2	
Servo 2	Roll	D3	

7.3.7.9 TRI_COPTER

This mixer offers control of 3 motors which are led by throttle, roll and pitch and 1 servos for yaw control. In config.hpp set TRI_COPTER to true to enable.

Note: The yaw servo must be angled s few degrees to counter torque and have I-gain to hold heading and overcome any residual motor torque.

Figure 25 – TRI_COPTER

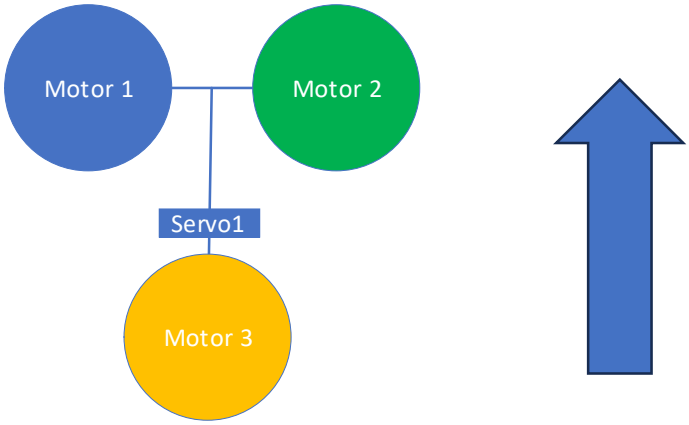


Table 14 – Tricopter Actuators

Actuator	Function	IO Pin	Comment
Motor 1	Throttle, roll, pitch	D0	
Motor 2	Throttle, roll, pitch	D1	
Motor 3	Throttle, roll, pitch	D2	
Motor 4	Unused	D3	Must declare as LEDC channel pairs share a common timer which prevents different PWM frequencies i.e. motor refresh and servo refresh.
Servo 1	Yaw	D8	

7.4 Section 4: Output Assignment

7.4.1 Model Outputs

In this section you must correctly assign the servo/motor outputs for the model type chosen. By doing this you map the physical output pins on the board. See 7.3 Section 3: Model Type to see correct motor and servo requirements for each model type. In the example below PLANE_FULL_HOUSE is used and maps 4 servos with 2 motors.

```
// PlaneFullHouse
static constexpr uint8_t SERVO_PINS[] =
{
    ESP32S3.OUTPUT_1,   // Servo 1 - e.g. left aileron  (PlaneFullHouse)
    ESP32S3.OUTPUT_2,   // Servo 2 - e.g. right aileron (PlaneFullHouse)
    ESP32S3.OUTPUT_3,   // Servo 3 - e.g. elevator      (PlaneFullHouse)
    ESP32S3.OUTPUT_4,   // Servo 4 - e.g. rudder        (PlaneFullHouse)
};

static constexpr uint8_t MOTOR_PINS[] =
{
    // PlaneFullHouse expects two motors here even when not physically fitted
    // as the code support differential thrust.
    ESP32S3.OUTPUT_5,   // Motor 1 - e.g. left/single motor  (PlaneFullHouse)
    ESP32S3.OUTPUT_6,   // Motor 2 - e.g. right motor (PlaneFullHouse)
};
```

7.4.2 Servo/Motor/Pass Through Refresh Rate

To optimise performance of you model you will need to set servos and esc rates to their highest possible. Always consult the manufacturers manual/instructions on maximum refresh rates as you may permanently damage servos by driving them with too higher rate. In Config.hpp find the following lines of code and update to suit your radio equipment:

Figure 26 - Servo/Motor Refresh Rate Settings

```
static constexpr LedcServo::RefreshRate SERVO_REFRESH_RATE = LedcServo::RefreshRate::IS_150Hz;
static constexpr LedcServo::RefreshRate MOTOR_REFRESH_RATE = LedcServo::RefreshRate::IS_150Hz;
static constexpr LedcServo::RefreshRate PASS_THROUGH_REFRESH_RATE = LedcServo::RefreshRate::IS_50Hz;
```

7.4.3 SERVO_REFRESH_RATE

The ESP32 LEDC PWM timer peripheral is set to 14-bit resolution. This allows a range of servo PWM/PPM refresh rates to be set from 50Hz up to 2KHz and therefore allows use of analogue or digital servos. Check the manufacturers data on your servos and set the appropriate refresh rate for your servos via the RefreshRate::IS_xxxHz enumeration.



CAUTION: Setting a servo refresh rate higher than the manufacturer recommends may prevent them from working correct and even damage them.

Due to how the LEDC PWM peripheral works, the lower the refresh rate results in lower resolution output for the servo i.e. lower resolution equals fewer servo steps over full travel of the servo. Table 15 details the servo resolution relationship and clearly shows that digital servos will offer better precision.

Table 15 - Servo Refresh Verses Resolution

RefreshRate Enumeration	Resolution/Servo Steps
IS_50HZ	819
IS_100HZ	1638
IS_150HZ	2458
IS_200Hz	3276
IS_250HZ	4096
IS_300HZ	4915
IS_350Hz	5461
IS_ONESHOT125 (2KHz implementation)	4096 (ESC output)

7.4.3.1 IS_ONESHOT125

When this is enabled motor outputs will use the Oneshot125 protocol as found on BLHeli ESCs. The oneshot125 protocol is a fast 4KHz PWM/PPM output with pulse width ranging from 125-250us.

This protocol allows faster and more accurate updates to the motors for multicopters, and when used for differential thrust aircraft and may increase stability in prop-hanging type manoeuvres.

Note: While Oneshot125 can output at up to ~4KHz PlainFlightController only outputs at 2KHz, this is done as the maths for the PWM/PPM works out nicely at ~12-bit resolution (4096 steps). Also, the main flight controller loop runs at 1KHz so there is no benefit to update above 1Khz, or at the maximum of 4KHz.

Note: BlHeli ESCs may have had different end points set. Either use the BlHeli App to configure or adjust the ONESHOT125_MIN_TICKS and/or ONESHOT125_MAX_TICKS constant expressions in LedcServo.hpp.

Note: There is no RC interpolation within PlainFlightController, as a result you will hear ~143Hz noise within the motors when moving the Tx sticks. Do not be alarmed this is just the ESC's adjusting the speed of the motors to suit the newest RC commands.

7.4.3.2 PassThroughStruct

The ESP32-S3 has 8 PWM channels that are used for servo/ESC outputs. If your chosen model type does not use all 8 outputs, then the remaining can be used to pass through raw transmitter channels for undercarriage or lights etc. PLANE_FULL_HOUSE leaves 2 spare channels for pass through. The below code example uses these spare channels to pass through AUX Tx channels for undercarriage and lights.

```
static constexpr PassThroughStruct PASS_THROUGH_PINS[] =
{
  //Output Pin to use,  Rx channel to assign
  {ESP32S3.OUTPUT_7,    RcChannelName::AUX5},  // E.g. Gear
  {ESP32S3.OUTPUT_8,    RcChannelName::AUX6}   // E.g. Lights
};
```

7.5 Section 5: RC Function Assignment

Assigns each switch/control function to a receiver channel position. Throttle, roll, pitch and yaw are fixed and are not assigned here. Arm and Mode channels are mandatory and must be set to a distinct auxiliary channel with AUX1 and AUX2 set at the default channels for these functions.

Flaperons, heading hold and prop-hang must be set to NONE while their corresponding feature flag (in SECTION 7 are not set. When the feature is enabled, set the matching constant to a distinct auxiliary channel that you wish to use.

```
static constexpr RcChannelName ARM_CHANNEL          = RcChannelName::AUX1;
static constexpr RcChannelName MODE_CHANNEL         = RcChannelName::AUX2;
static constexpr RcChannelName FLAPS_CHANNEL        = RcChannelName::NONE;
static constexpr RcChannelName HEADING_HOLD_CHANNEL = RcChannelName::NONE;
static constexpr RcChannelName PROP_HANG_CHANNEL    = RcChannelName::NONE;
```

7.6 Section 6: Output Configuration

7.6.1 REVERSE_SERVO_x

These settings are used if you find gyro/levelled corrections are operating in the wrong sense after setting **Error! Reference source not found. Error! Reference source not found.**. Needing to reverse servos entirely depends on how your model has been mechanically configured.



Note: After setting servo reversing check that your transmitter stick inputs operate in the correct sense. If they do not, then reverse the stick direction within the transmitter software.

7.6.2 EXTEND_SERVO_TRAVEL_RANGE

When WiFi GUI trims have been used to offset servo centre position set EXTEND_SERVO_TRAVEL_RANGE to true. This extends servo travel beyond the standard 1.0–2.0 ms pulse range in order to regain the full range of servo travel. When true, travel is extended up to 0.8–2.2 ms.



CAUTION: Ensure your servos can operate with the extended travel range or permanent damage may be done to your servos.

7.6.3 CALIBRATE_ESC



CAUTION: ALWAYS REMOVE PROPELLER(S) WHEN CALIBRATING ESC(S). YOU RISK SERIOUS INJURY IF YOU DO NOT.

When set to true, at power on the flight controller will output maximum throttle value and hold for CALIBRATE_HOLD_TIME, then set minimum throttle value. This should calibrate the ESC(s) full range to match the flight controller output.

Note: When calibrating you will need to use the flight battery to power the ESC(s) as the 5V USB supply may not power the ESC. Depending on your ESC(s) you may need to reduce CALIBRATE_ESC_DELAY to prevent it from entering programming mode.

Once calibrating has finished, code execution is halted. You will need to set CALIBRATE_ESC to false and reprogram the flight controller.

Note: Always calibrate multicopter ESCs before the first flight.

Note: For the 3.3V version avoid powering ESC via the USB connection – you may damage your laptop/USB power supply.

7.7 Section 7: Feature Flags

7.7.1 USE_FLAPS (Fixed Wing Only)

When using PLANE_FULL_HOUSE or PLANE_FULL_HOUSE_V_TAIL that has two aileron servos you can enable the flaperons function. Set USE_FLAPS constexpr to true if you wish to use flaperons.

Note: do not enable for multicopters.

7.7.2 USE_DIFFERENTIAL_THRUST (Fixed Wing Only)

If your fixed wing model has twin motors then by uncommenting this allows you to use differential thrust via rudder Tx commands. Differential throttle will be gyro stabilised in rate and self-levelled modes.

Note: do not enable for multicopters.

7.7.3 USE_HEADING_HOLD (Fixed Wing Only)

When enabled a heading hold function will be mapped to the rudder. This is done by applying I-gain to the PIDF yaw calculation when heading hold switch is enabled.

The intention of heading hold is to assist tracking during manoeuvres such as run off ground take off's, vertical climbs, loops and prop hanging.

Note: Comment out if you have a transmitter with only 6 channels.
Note: do not enable for multicopters as it is automatically enabled.

7.7.4 USE_LOW _VOLTS_CUT_OFF

When enabled battery voltage will be monitored and the system will react to low battery volts by actively reducing the throttle output(s) and ultimately cutting the throttle.

Motor power reduction starts when the battery falls to 3.4V per cell, and complete throttle cut off occurs at 3.3V per cell. If you are using lithium-ion cells then these values will need to be changed to what the manufacturer recommends and will be approximately 2.7V and 2.5V respectively.

This option is only recommended when using BlHeli type ECSs where low voltage detection is not part of the ESC software.

Note: For this to operate correctly you must have the potential divider hardware connected to D10 analogue input.

7.7.5 USE_EXTERNAL_LED

When set to true a second instance of LED will be instantiated on the defined IO pin. This will be used to indicate the flight controller mode and is useful if the flight controller is hidden inside a fuselage.

The LED can be configured to be sink or source configuration, this is done at the point the where the LED object is declared in FlightControl.hpp.

7.7.6 USE_ACRO_TRAINER

For fixed wing model types, when set to true the acro trainer mode will be use instead of self-levelled mode. This mode self-levels when pitch and roll sticks are centred. When pitch and/or roll are not centred rate mode is used. This mode is similar to the horizon mode of other flight controllers. It allows aerobatics but hands off sticks will perform an auto recovery to level.

It is essential your Tx trims are set to neutral, or this mode is unlikely to work. The model should be trimmed mechanically to fly straight and level.

Note: This mode is always used for multicopters.

7.7.7 HAS_TELEMETRY

When using a radio protocol with telemetry (CRSF) both battery and GPS data can be downlinked to your transmitter. Setting this to true enables telemetry to be sent for battery, enabling GPS in section 7 enables GPS data to be downlinked.

7.7.8 USE_PROP_HANG_MODE (Fixed Wing Only)

When set to true Tx an auxiliary channel will be used to activate the self-levelled prop-hang mode where rudder and elevator are used to keep the models nose vertical.

Note: your model must be capable of prop-hanging in the first place !

There are several settings to USE_PROP_HANG_MODE which are described below. These will hopefully be rationalised in future releases.

7.7.8.1 PROP_HANG_TAIL_SITTER_MODE

Set to true for tail-sitter mode where roll stick commands the model's yaw, and yaw stick commands the models roll when prop-hang mode is enabled.

7.8 Section 8: Optional GPS

When using a receiver protocol that supports telemetry (CRSF) a GPS can be added to downlink data to your transmitter. The GPS telemetry data includes ground speed, height, number satellites and coordinates.

7.8.1 GNSS_TYPE

Supported GPS types are CASIC or Ublox, and you will have to select one of the following options:

Figure 27 - GNSS Type Options

GNSS Type
CASIC
UBX
NONE

7.8.2 GENERATION

When using Ublox GPS you will need to set the generation type. The following options are available:

Figure 28 - Ublox Generation Options

Ublox Generation
UBX_M6_MINUS
UBX_M7_M8

UBX_M9_PLUS

7.9 Section 9: Flight Characteristics

7.9.1 GYRO_RATE

Defines the maximum rate at which the aircraft can rotate.

Figure 29 - Gyro Rate Options

Degrees Per Second	Comment
IS_250_DEGS_SECOND	Ideal for sport and precision aerobatic flying
IS_500_DEGS_SECOND	Use for multicopters and 3D fixed wing models

Only set IS_500_DEGS_SECOND if your model can achieve more than 250 degrees/second rates of rotation, as this setting will lower resolution of the sensor output, and may reduce performance for models that cannot manoeuvre at these rates.

7.9.1.1 ACRO_TRAINER_LEVEL_RATE

Due to the way acro trainer works we may get a large step demand when sticks are centred, and the model is at an extreme angle. This would result in the maximum rate demand being applied and can lead to levelling overshoot particularly on fast acting multicopters. This parameter instead allows us to fine tune how quickly we return to level in degrees per second and is generally set at a lower rate than your flight control rates. Consider it to be a levelling strength parameter.

Caution: Do not exceed and stay well below the degrees per second you have set for the gyro.

Caution: Do not exceed your set control rates degrees per second

Caution: Setting this too high can lead to flight gains over reacting and the craft returning to level faster than your set control rates and gyro degs/sec setting which can cause the Madgwick filter to loose true level – particularly on fast acting multicopters.

Caution: Do not exceed your models maximum degrees per second capabilities or you will get I-gain build up and overshoot level.

7.9.1.2 REVERSE_PROP_HANG_x_CORRECTIONS

When prop-hang mode is active, if you find that rudder or elevator is correcting in the wrong sense then set the appropriate constexpr to true to reverse the corrections.

7.9.1.3 PROP_HANG_REVERSE_x_DEMAND

When prop-hang mode is active, if you find that yaw/roll controls are working in the correct sense, but the Tx commands are reversed, you can use these settings to reverse the Tx commands for the appropriate control surface.

This problem is generally associated with having tail sitter mode enabled. This will hopefully be rationalised in future releases.

7.9.1.4 IMU_PLUS_X/Y

The IMU board can now be orientated in any of the 24 possible orientations. This is done by specifying which direction the positive X and positive Y axis of the accelerometer are facing. Set IMU_PLUS_X and IMU_PLUS_Y to indicate which direction they are facing on the aircraft. Table 16 - IMU Orientation Setting

details the possible orientation settings, while Figure 30 - IMU Orientation Example details a common example.

Table 16 - IMU Orientation Setting

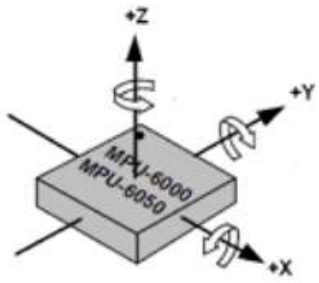
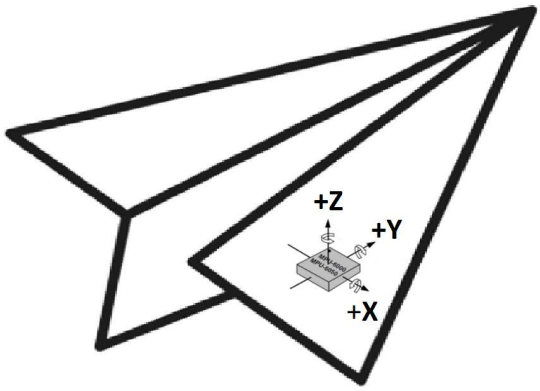
MPU6050 Accelerometer +ve Axis	IMU_PLUS_X/Y Settings
	FRONT
	BACK
	LEFT
	RIGHT
	UP
	DOWN

Figure 30 - IMU Orientation Example

IMU Orientation On Aircraft	Code Setting
	IMU_PLUS_X = RIGHT
	IMU_PLUS_Y = FRONT

7.9.1.5 TX_DEAD_BAND_xxx

Radio control transmitters may have a bit of jitter or inaccuracy around centred positions. This dead band can be used to mask this jitter or inaccuracy.
If you find that certain control surfaces are drifting in rate mode with Tx sticks and trims centred, then try increasing the dead band.

Note: Control surface drift could also be caused by a poor power on gyro calibration i.e. model was moving during calibration, but not by enough to trip a calibration reset.

7.10 Section 9: Multicopter Motor Settings

7.10.1.1 IDLE_UP_VALUE (Multicopters Only)

When you arm a multicopter the motors will spin up to a speed defined by IDLE_UP_VALUE. You may need to tune this value (with propellers removed) to a speed that makes the motors spins with reasonable energy but not to the point where the model is on the verge of trying to lift off from the ground.

Adjust this if IDLE_UP_VALUE speed is too low/high, for best performance it needs to be set to a speed where the model is a few throttle clicks away from lifting off.

7.10.1.2 MIN_THROTTLE_VALUE (Mutlicopters Only)

This is the absolute minimum motor speed to prevent motors from stopping/stalling. Adjust this if MIN_THROTTLE_VALUE speed is too low/high. This setting allows the PID controller to work around the IDLE_UP_VALUE to try and give an ‘air mode’ locked in feeling when at minimum throttle.

8. Transmitter Configuration

To obtain full functionality of PlainFlightController you will need an 8-channel transmitter. Channels 1 to 4 are your primary flight controls of throttle, aileron, elevator, rudder and need to be set to 100% travel. Auxiliary channels are configured for switch inputs with an example shown in Figure 31, switches must also be set for 100% travel.

Figure 31 - Typical Tx Switch Setup



Table 17 details the requirements of these switch inputs so that flight controller functions operate correctly.

Table 17 - Tx Switch Requirements

Switch	Type	Tx Channel Example	Usage
Arm Switch	2 position latching	AUX1	>1500us = armed
Mode Switch	3 position	AUX2	Fixed wing: <1250us = pass through >1250us & <1750us = rate mode >1750us = self-levelled or acro trainer mode Multicopters: <1250us = rate mode >1250us & <1750us = acro trainer >1750us = self-levelled mode
Flaperon Switch	3 position	AUX3	<1250us = no flaperons >1250us & <1750us = flaperons 1 >1750us = flaperons 2
Heading Hold Switch	2 position (latching if preferred)	AUX4	>1500us = heading hold active.
Prop-hanging Switch	2 position (latching if preferred)	AUX5	>1500us = self-level prop hang mode

Note: Flight controller will not arm if throttle is high.
Note: Failsafe will be entered if Tx reception is not initiated or is lost.

8.1 Trims

It is essential that your transmitter trims are set to their neutral position, if they are not when you enter rate or levelled modes the flight controller will see the trim(s) offset as a demand and make the model pitch, roll, or yaw.
You will need to set servo control arms/horns perpendicular to the servo body and adjust the linkages mechanically to make the model fly straight and level.

8.2 Arming Switch

Like any flight controller it will need to be armed before the motor(s) will operate. Whilst disarmed all servos will operate in pass through mode i.e. no stabilisation will be applied. Arming is controlled via an auxiliary channel, usually AUX1.

8.3 Mode Switch

Once armed the flight controller operates in one of 4 main modes of pass-through, rate, self levelled and acro trainer. These modes are controlled via an auxiliary channel usually AUX2 and a 3 position switch.

8.3.1 Pass Through Mode

When in pass-through mode the transmitter commands are passed directly through to the servos and motor(s). Should you experience any control problems in flight switch back to pass-through mode to disable all PIDF calculations.

8.3.2 Rate Mode

When in rate mode the 3 axis gyro is used to apply stabilisation to roll, pitch and yaw axes. Transmitter stick commands are converted to degrees/second and PIDF calculations are applied for each axes based on stick commands and gyro data, these calculations depend on the gains set for each corresponding axis.

Note: Setting gains too high will likely make the model suffer from oscillations and could make it uncontrollable.

8.3.3 Self-Levelled Mode

Self-levelled mode using both the gyro and accelerometer; this mode combines data via a fusion algorithm known as a Madgwick filter which computes the IMUs attitude relative to gravity. Transmitter stick commands are converted to a desired angle in degrees and PIDF calculations are applied for pitch and roll axes based on stick commands and IMU attitude, these calculations depend on the gains set for each corresponding axis. Aelf-Levelled mode will not allow the pilot to exceed a predefined pitch or roll angle.

8.3.4 Acro Trainer Mode

Settable for fixed wing model types, when set to true in Config.hpp the acro trainer mode will be used instead of self-levelled mode. This mode self-levels the model when pitch and roll sticks are centred but when pitch and/or roll are not centred rate mode is used so rolls and loops can be achieved. This mode is similar to the horizon mode of other flight controllers. It allows aerobatics to be performed, but hands off sticks will perform an auto recovery to level flight.

It is essential your Tx trims are set to neutral, or this mode is unlikely to work. The model should be trimmed mechanically to fly straight and level.

Note: This mode is always used for multicopters.

8.4 Flaperons Switch (Now channel selectable)

When enabled in the software a 3-position switch driven by a channel will give either, no flaperons, half flaperons, or full flaperons. Currently half flaperons result in 25% of servo travel and full flaperons 50% of servo travel.

8.5 Heading Hold Switch

When enabled in the software a 2-position switch driven by an auxiliary channel will give a heading hold function to the rudder (yaw axis). This is done by applying I-gain to the PIDF yaw calculation when heading hold switch is enabled.



Caution: Do not attempt to fly with this permanently enabled as the heading hold will fight an aileron turn.

The intention of heading hold is to assist tracking during manoeuvres such as run off ground take off's, vertical climbs, loops, prop hanging and knife-edge flight. The user will need to switch this heading hold function on and off as and when needed during flight manoeuvres.

8.6 Prop-hanging Mode Switch

Self-levelled prop-hanging mode using both the gyro and accelerometer; this mode will try to keep the models nose pointed vertically in prop-hang. Uses the Madgwick filter which computes the IMUs attitude relative to gravity. Transmitter stick commands are converted to a desired angle in degrees and PIDF calculations are applied for pitch and yaw axes based on stick commands and IMU attitude, these calculations depend on the gains set for each corresponding axis. Note that this mode can also be used for tail-sitter VTOLs.

Note: Prop-hanging mode overrides passthrough, gyro rate and self-levelled modes.

Note: You can set a tail sitter mode where yaw and roll controls are swapped when in prop-hang mode.



Caution: Your model must be capable of prop hanging in the first place for this mode to be able to work.

9. How To Test

Following your build of your PlainFlightController controller you will need to test the following items. Testing is done via setting the DEBUG_ constant expressions in Config.hpp to true. Only set one DEBUG_ constant expression to true at one time or the data displayed will be unreadable.

Note: If at any time your flight controller refuses to program, remove USB from your computer, press and hold the boot button on the XIAO ESP32 then insert the USB and after 1 second of it being powered release the boot button. Now reprogram.

Note: Always set DEBUG_ items to false before flying as the data output may slow the control loop and affect flight performance.



Caution: Ensure propeller(s) are removed when connecting USB with flight battery in circuit. Inserting/removing the USB can cause the XIAO to reset and result in undefined behaviour of motor outputs i.e. motors may operate !



Caution: The 1s circuit will back feed 5V USB power to the ESC's, ensure ESC's can tolerate 5V and remove propeller(s) and note that inductive spikes from motors may damage the connected USB device.

9.1.1 LED Status

The LED indicates the status of the flight controller via different flash sequences, these are detailed in Table 18.

Table 18 - LED Sequences

Mode	Sequence	Neopixel Colour
Initialising/calibrating	Fast on/off flash	White
Failsafe	Very fast on/off flash	Purple
Disarmed	Slow 1 second on/off flash	Green
Armed & passthrough mode	1 quick flash with following pause	Red
Armed & rate mode	2 quick flashes with following pause	Red
Armed & self-levelled mode	3 quick flashes with following pause	Red
Armed & acro trainer mode	4 quick flashes with following pause	Red
Waiting to disarm	5 quick flashes with following pause	Blue
WiFi Configuration	6 quick flashes with following pause	Yellow
Fault	7 quick flashes with following pause	Purple
Armed & prop-hang mode	8 quick flashes with following pause	Red

Verify that your Armed switch and Mode switches operate and give all LED sequences shown in Table 18.

Note: There is no flash sequence for heading hold as it is a transparent command used in rate and self-levelled modes.

9.2 DEBUG

Found in InternalConfig.hpp the debug defines are used to help problem solve the flight controller. It is recommended that each of these is enabled one at a time to confirm correct operation of the flight controller following its construction. Use Arduino IDE 'Serial Monitor' to view the data, Table 19 details what debug data you can view.

Contained in InternalConfig.hpp you will find a number of DEBUG SETTINGS. Set one of these to true to debug a particular area. Always ensure you set them back to false and reflash the flight controller once you are done.

Section 9 How To Test gives greater detail on each debug setting and the data output.

Table 19 - Debug Data Output

Debug	Description
DEBUG_SBUS	Outputs raw Sbus data. Confirms correct wiring and that Sbus data is being received.
DEBUG_RC_DATA	Outputs refactored Sbus data that the flight controller uses.
DEBUG_LOOP_RATE	Outputs the loop rate as float in seconds. You should see a value of 0.001 (up to 0.001002 is normal).
DEBUG_BATTERY_MONITOR	Outputs battery volts as a float and other associated data.
DEBUG_MADGWICK	Outputs 3-axis IMU data from the Madgwick filter. This data is used for self levelled flight mode.
DEBUG_GYRO_CALIBRATION	Outputs data relevant to gyro calibration. If you have a noisy gyro then CALIBRATE_MAX_MOTION may need to increase.
DEBUG_MPU6050	Outputs 6-axis data from the gyro. Can be used to confirm the gyro is connected and working correctly.
DEBUG_ OUTPUT	Outputs servos, motors and pass through channels as signed integers. May help with debugging cross mixing of channels.
DEBUG_GPS	Outputs GPS data and can assist in proving correct operation of the connected GPS.

9.2.1 DEBUG_GYRO_CALIBRATION

In Config.hpp set DEBUG_GYRO_CALIBRATION to true and program your flight controller.

Note: When the flight controller gyro calibrates at power on, small or sustained movements will cause it to halt and to retry calibration. Calibration will not complete until the flight controller is still.

Place the flight controller on a flat surface, ensure it is still and plug the USB cable into your flight controller and computer.

On Arduino's Serial Monitor you should see messages similar to:

```
Variance (LSB^2): 101
Motion detected – restarting calibration
Variance (LSB^2): 85
Variance (LSB^2): 85
Calibration complete...
x: 65    y: 16    z: -55
```

Note: you may see 'Motion detected – restarting calibration' messages if your flight controller was not still at power on. Wind or unstable work surfaces can prevent calibration from completing.
Note: Your X, Y and Z calibration values will be different.

If you do not see these messages, then you may have a power or I2C wiring issue between the ESP32 and the MPU6050.

Alternatively, if you have a noisy gyro you may need to increase CALIBRATE_MAX_MOTION to a larger value. If you do need to increase this value, consider changing the MPU6050 for another. A lot of these MPU6050's are Chinese clones and the quality varies.

Note: The calibration state is only exited on successful calibration. Should you be moving the flight controller or model during calibration then it will not exit, and the flight controller cannot be armed. Calibration state is indicated by a constant 250ms second LED flash.

9.2.2 DEBUG_MPU6050

In Config.hpp set DEBUG_MPU6050 to true and then reprogram your flight controller.

The Serial Monitor should now be continually displaying gyro and accelerometer data. Confirm that rotating/tilting your flight controller in X, Y and Z axes causes the corresponding axis data to change. When still your Arduino Serial Monitor you should display something similar to:

```
gx:0.02  gy:0.03  gz:0.03  ax:0.05  ay:-0.05  az:0.99
gx:0.02  gy:0.03  gz:0.03  ax:0.05  ay:-0.05  az:0.99
gx:0.02  gy:0.03  gz:0.03  ax:0.05  ay:-0.05  az:0.99
gx:0.02  gy:0.03  gz:0.03  ax:0.05  ay:-0.05  az:0.99
```

Note: All data should change as you tilt and rotate the flight controller.

In the unlikely case that some data is stuck and does not change you likely have a bad MPU6050.

9.2.3 DEBUG_MADGWICK

If the gyro tests in 9.1.1 and 9.2.2 are successful, then there is no need to perform this test. However, it may be of interest to you.

In Config.hpp comment set DEBUG_MADGWICK to true then reprogram your flight controller.

The Serial Monitor should now be continually displaying roll and pitch data. You will only see yaw data if you have USE_PROP_HANG_MODE set to true.

Confirm that rotating/tilting your flight controller in X, Y and Z axes causes the corresponding axis data to change between 0 and +-180 for roll, 0 and +- 90 degrees for pitch and yaw (asin or atan usage determines range). On your Arduino Serial Monitor you should see data similar to:

```
Roll: -0.46, Pitch: -1.75
Roll: -0.46, Pitch: -1.74
Roll: -0.46, Pitch: -1.74
Roll: -0.47, Pitch: -1.78
```

9.2.4 DEBUG_LOOP_RATE

There is no need to perform this test unless you have modified the software for your own purposes and wish to check that your changes have not affect the 1KHz loop rate.

In Config.hpp set DEBUG_LOOP_RATE to true then reprogram your flight controller.

The Serial Monitor should now be continually displaying floating-point numbers. The 3 columns of data represent the stable loop time, the actual execution time, and the running average loop time. All data is shown in micro seconds (0.001000s = 1.000ms = 1000us = 1.0KHz). The stable loop time may fluctuate by 2us (0.001000 to 0.001002), this is ok.

0.001000	0.000944	0.000864
0.001001	0.000925	0.000864
0.001001	0.000940	0.000864
0.001001	0.000865	0.000864

Note: 1/0.001000 = 1000Hz (1KHz) loop rate.

9.2.5 DEBUG_RX

In Config.hpp set DEBUG_SBUS to true then reprogram your flight controller.

With you transmitter on and having bound the transmitter to your receiver the Serial Monitor should now display 8 channels of Sbus data (9 if USE_PROP_HANG_MODE is enabled), followed by the lost frame and failsafe bits:

172	992	992	985	172	992	992	992	0	0
172	992	992	985	172	992	992	992	0	0
172	992	992	985	172	992	992	992	0	0
172	992	992	985	172	992	992	992	0	0

Note: Channels 8 to 16 will only be displayed if enabled in the software.

Moving your transmitter sticks should cause corresponding channel data to change. Turning off your transmitter should result in lost frame and failsafe bits at the end of the coms data line to be set to ‘1’.

If no data is displayed, then check your wiring and that your receiver is bound to your transmitter.

If data is displayed but does not change, ensure you have setup your transmitter correctly to output 8 channels for pitch, roll, yaw, throttle, aux1, aux2, aux3 and aux4 (9 channels when using prop-hang mode).

9.2.6 DEBUG_RC_DATA

In Config.hpp set DEBUG_RADIO_COMMANDS to true then reprogram your flight controller.

In Serial Monitor verify operation by moving the sticks or turning switches on/off, the outputs should be as described by Table 20.Typical data will look like:

```
pitch: 0 roll: 0 yaw: 0 throttle: -1024 flaps: -1024 DISARMED mode: 1 HeadingHold: OFF PropHang: OFF
pitch: 0 roll: 0 yaw: 0 throttle: -1024 flaps: -1024 DISARMED mode: 1 HeadingHold: OFF PropHang: OFF
pitch: 0 roll: 0 yaw: 0 throttle: -1024 flaps: -1024 DISARMED mode: 1 HeadingHold: OFF PropHang: OFF
```

Note: When prop-hang mode and/or heading hold are enabled you will see these appear in the data.

Note: It is important to verify that your switches are operating correctly so correct flight modes can be set.

Table 20 - Commands

Command	Outputs
Armed	0 and 1
Mode	0, 1, and 2
Aux1 (Flaperons)	0, 1, and 2
Aux2 (Heading Hold)	0 and 1
Aux3 (Prop-Hang)	0 and 1
Throttle	Is a signed number that varies between SBUS::MIN_NORMALISED_US and SBUS:: MAX_NORMALISED_US.
Aileron	When disarmed is a signed number that varies between SBUS::MIN_NORMALISED_US and SBUS:: MAX_NORMALISED_US. When armed values depend on flight mode and your set rates or max angles.
Pitch	When disarmed is a signed number that varies between SBUS::MIN_NORMALISED_US and SBUS:: MAX_NORMALISED_US. When armed values depend on flight mode and your set rates or max angles.
Yaw	When disarmed is a signed number that varies between SBUS::MIN_NORMALISED_US and SBUS:: MAX_NORMALISED_US. When armed values depend on flight mode and your set rates or max angles.

9.2.7 DEBUG_BATTERY_MONITOR

You should only need to use this if you are unable to use the WiFi web configurator as battery voltage calibration can be done on it.

Either way you will need a multimeter to measure DC voltage of your flight battery. If motor(s) is connected ensure your propeller(s) are removed.

In Config.hpp set DEBUG_BATTERY_MONITOR then reprogram your flight controller.

Serial Monitor will now be displaying several items related to the battery monitor, these being the raw ADC value, number of cells detected, battery scaler, and battery voltage. Use your multimeter to verify that the battery voltage given by the flight controller is the same as it reads on your multimeter. It will likely be different due to resistor tolerances, if it differs by more than 0.1V then you can calibrate it via the WiFi web configurator or manually edit the BATTERY_SCALER in InternalConfig.hpp. Note if you manually edit it in the code then you will need to erase all flash when programming for it to take effect. Note the calculated voltage is heavily filtered and will slightly lag the battery voltage.

```
ADC: 2141, Cells: 2, Scaler:0.00378, volts:8.10
ADC: 2144, Cells: 2, Scaler:0.00378, volts:8.10
ADC: 2147, Cells: 2, Scaler:0.00378, volts:8.10
ADC: 2141, Cells: 2, Scaler:0.00378, volts:8.10
```



CAUTION: If you are powering from the flight controller from 1s 3.3V lipo battery, do not plug in the flight battery and USB in at the same time, as the USB will feed 5V to the lipo battery. For 1s lipo powered system calibrate the battery voltage via the USB 5V supply.



Caution: Ensure propeller(s) are removed when connecting USB with flight battery in circuit. Inserting/removing the USB may cause the flight controller to reset and output an undefined stated to an ESC.



Caution: The 1s circuit will back feed 5V USB power to the ESC's, ensure ESC's can tolerate 5V and remove propeller(s).

9.2.8 DEBUG_OUTPUT

There is no need to perform this test unless you have modified the software for your own purposes and wish to check that your servo mixing output are working as you intended.

In InternalConfig.hpp set DEBUG_OUTPUT to true then reprogram your flight controller.

Data is displayed as ‘timer ticks’ required to reproduce the 1ms to 2ms pulse. Data range will depend on the refresh rate you have chosen. Section 7.4.2 Servo/Motor/Pass Through Refresh Rate indicates the relation of servo resolution, or range of timer ticks you should expect to see.

```
Servo 1: 3510
Servo 2: 3510
Servo 3: 3510
Servo 4: 3510
Motor 1: 4096
Motor 2: 4096
Pass Through 1: 3510
Pass Through 2: 3510
```

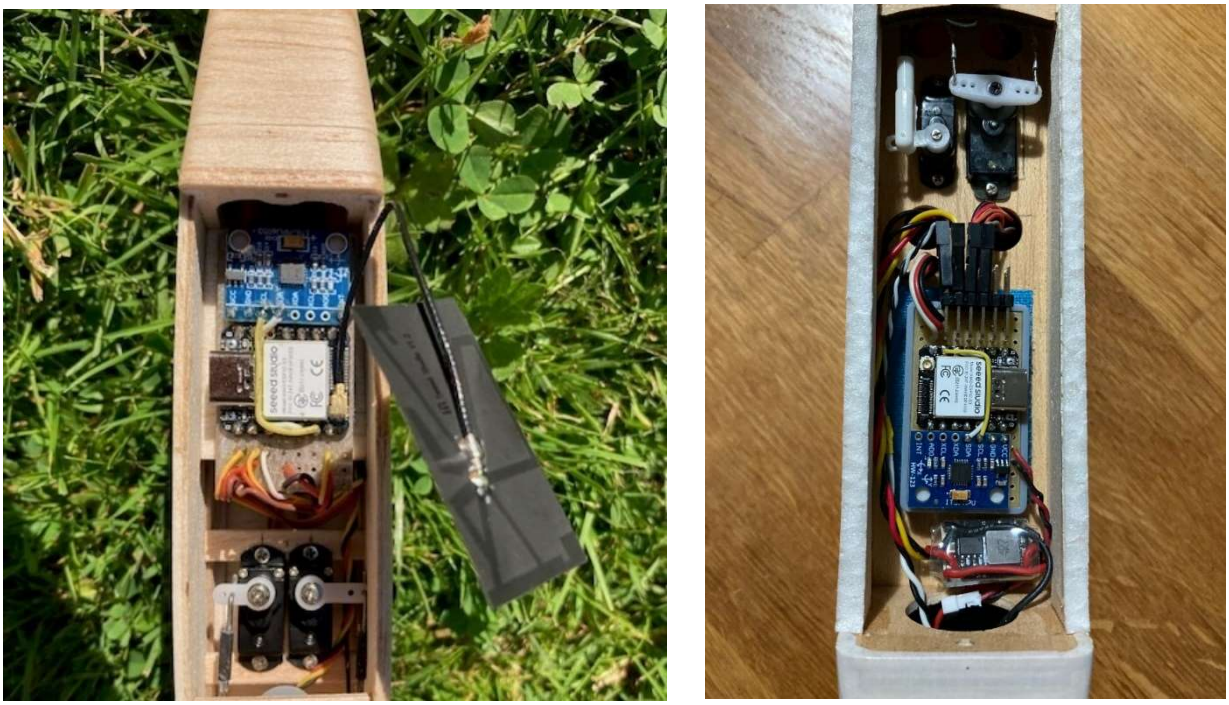
9.2.9 DEBUG_GPS

To assist with GPS connectivity this option can be enable and will output hour, minute, seconds, number satellites, latitude and longitude.

10. How To Install In Your Model

The flight controller can be mounted in any orientation. Its configuration is defined in Config.hpp and depending how your servos are mechanically coupled you may also need to configure pitch/roll/yaw corrections as described in section 7 How To Configure Your Model.

Figure 32 - Flight Controller Installation



The flight controller should be mounted on foam tape or Velcro to help absorb any vibrations and ensure that no wires or objects can vibrate or knock onto the flight controller when in flight. Ensure that the WiFi antenna is not surrounded by other wiring or metallic objects or you may experience connection issues. Be aware the WiFi aerial and connector has been found to be quite susceptible to damage from heavy arrivals so try to secure or stow it.

11. How To Tune

Firstly, if you have never tuned a flight controller or do not understand PIDF then seek assistance or training as this document assumes you have knowledge of flight controllers and understand PIDF control systems.

To tune your flight controller at minimum you will have to set rates and PIDF parameters for each axis. The following sections detail how this can be done.

11.1 WiFi Web Configurator

PlainFlightController V2.0.0 onwards supports a WiFi web-based configurator to allow easy tuning of flight parameters on any smart device with a web browser. This means configuration of the following parameters can be done when out flying your model without the need to completely reprogram the ESP32:

Table 21 - Web Configurable Parameters

Parameters	Comments
PIDF	Proportional, integral, differential and feed forward for roll, pitch and yaw axis.
Rates	Maximum degrees per second of rotation for roll, pitch and yaw axis.
Max Self-Levelled Angles	Maximum allowed bank angles for pitch and roll when in self-levelled mode.
Levelled Trim	Fine trim self-levelled flight with pitch and roll trims.
Servo Trims	Used to fine tune the servo horn to centre position. Not to be used for incorrect mechanical linkage adjustment – you will lose range of servo motion if not mechanically set correct.
Battery Scaling	Fine tune the battery voltage monitor.

Note: If you are unable to use the PIDFs, Rates and trim parameters in defines.h are used for the initial setup only. Changing these parameters and programming the ESP32 will not change flight settings as they did for previous versions of PlainFlightConroller. All changes must be made through the web interface to take effect.

11.1.1 How To Connect

To connect your smart device to your flight controller you must follow these easy steps:

1. Model is disarmed (LED slow 1 second on/off flash)
2. Throttle is moved to fully open position

When the model is disarmed and throttle set to maximum the WiFi access point is started by the ESP32, this is indicated by 5 repeating quick LED flashes. Once established, go to your WiFi settings on your smart device and you should see ‘PlainFlight’ appear in the list of WiFi providers.

On selecting ‘PlainFlight’, for the first ever connection you will be prompted to enter a password (12345678). WiFi connection can be instantaneous, or take up to ~15 seconds.
Should it not connect, close the throttle, and wait until ‘PlainFlight’ disappears from your smart device, then reopen the throttle to reinitialise.

When your smart device shows connection is established it should automatically open the web page. If not open your browse and it should open. In a case where it does not load then enter the address 192.168.4.1 and the configurator interface should load. Should it not load your virus checker may be blocking. The web interface has been tested on iPhone, iPad, Android phone, and Windows 10, with Chrome, Edge, and Safari browsers.

Table 22 - Web Configurator Information

Parameters	Data
WiFi Provider	PlainFlight
Password	12345678
Web Address	192.168.4.1

The following devices have been tested with the web configurator:

Table 23 - Tested Smart Devices/Browsers

Device	Browser	Comments
Laptop (Win 10/11)	Microsoft Edge	Works well
Iphone SE	Safari	Works well
Iphone SE	Chrome	Works well
Samsung Galaxy	Chrome	Works well
Ipad (~2020)	Safari	Works well

Note: I have found my dated iPhone to be unreliable in establishing a WiFi connection, but all other devices generally work first time. I have also found that the WiFi connector on the XIAO to be very fragile and can fail rendering the WiFi inoperable.

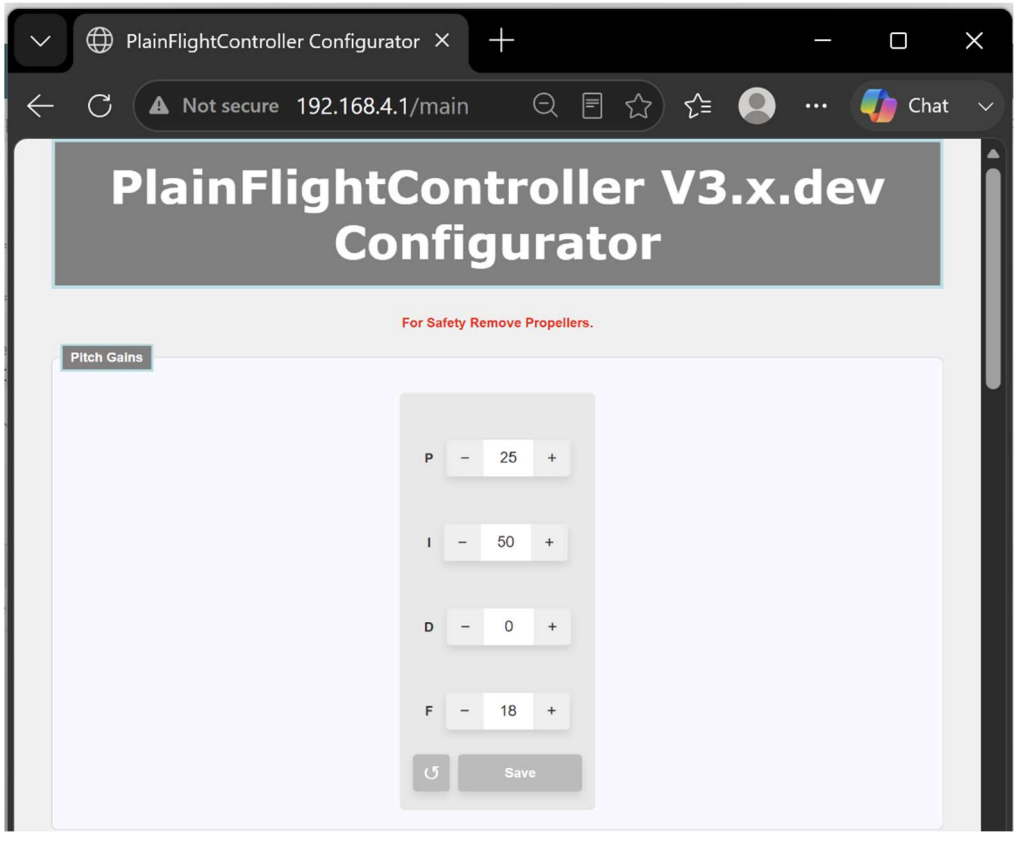
Caution: Ensure the WiFi aerial is connected to your ESP32-S3 or you may permanently damage it.

Caution: For safety it is strongly advised to remove all propellers before entering WiFi configurator mode.

Note: The password (12345678) is used to prevent unsecure network warnings from your smart device.
Note: Keep WiFi aerial away from other wiring or metal objects to ensure reliable connection.
Note: Keep your radio transmitter ~1m away from you model and smart device when using WiFi to help reduce any connection issues.

On successful connection you should be presented with the PlainFlightController Configurator as shown in Figure 33. The configurator is based on HTML forms for data entry, this breaks down data items into groups for easy data entry. Values shown in the web configurator on loading are those of your model it has connected to. When you edit values the original pre-edit value is displayed along with the max/min range that you can enter.

Figure 33 - Web Configurator On MS Edge Browser



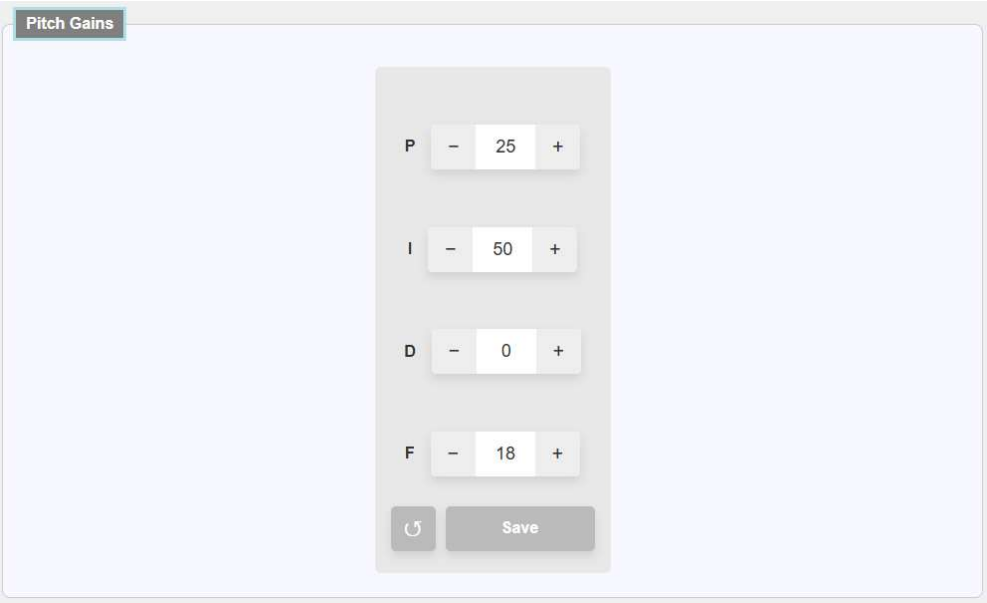
11.1.2 PIDF Configuration

Proportional, Integral, Differential, and Feed Forward for Pitch, Roll, Yaw axis can be set to allow easy tuning of your model's gain parameters.

To tune pitch, roll and yaw, you will be presented with three separate forms for Pitch Gain, Roll Gains, and Yaw Gains. The Pitch Gains example given in Figure 34 clearly shows P, I, D, and F that can be configured. Enter the required values you want then press the save button associated with the gains you have just changed.

On saving the form will reload and show the values saved.

Figure 34 - Pitch Gains Form



11.1.2.1 Rate Configuration

Rate Configuration allows you to set the maximum degrees per second that your model will rotate for Pitch, Roll and Yaw.

Note the configurator will not allow you to set beyond the maximum value you have set within the PlainFlightController code (250 or 500 degs/s). Also your model should be capable of rotating at the rates you are setting.

Enter the values you require, then press save.

Figure 35 – Rates Form

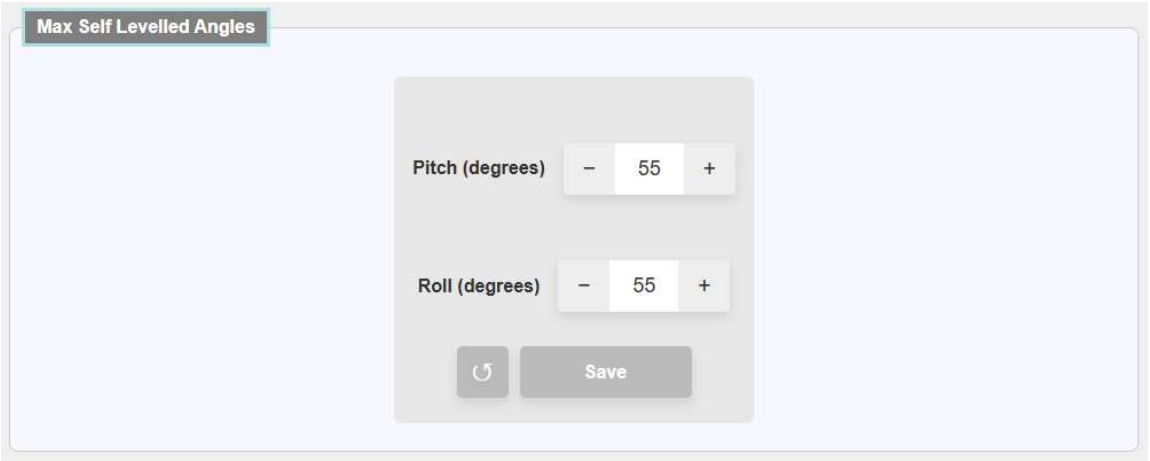


11.1.2.2 Max Self-Level Angle Configuration

This form allows you to set the maximum pitch and roll angles that are allowed when flying in self-levelled mode. Setting these too low will result in your model feeling unresponsive and may make turns impossibly large. It is recommended that you use value around 60 degrees.

On editing the values, press save to store the new settings.

Figure 36 - Max Self Levelled Angles Form



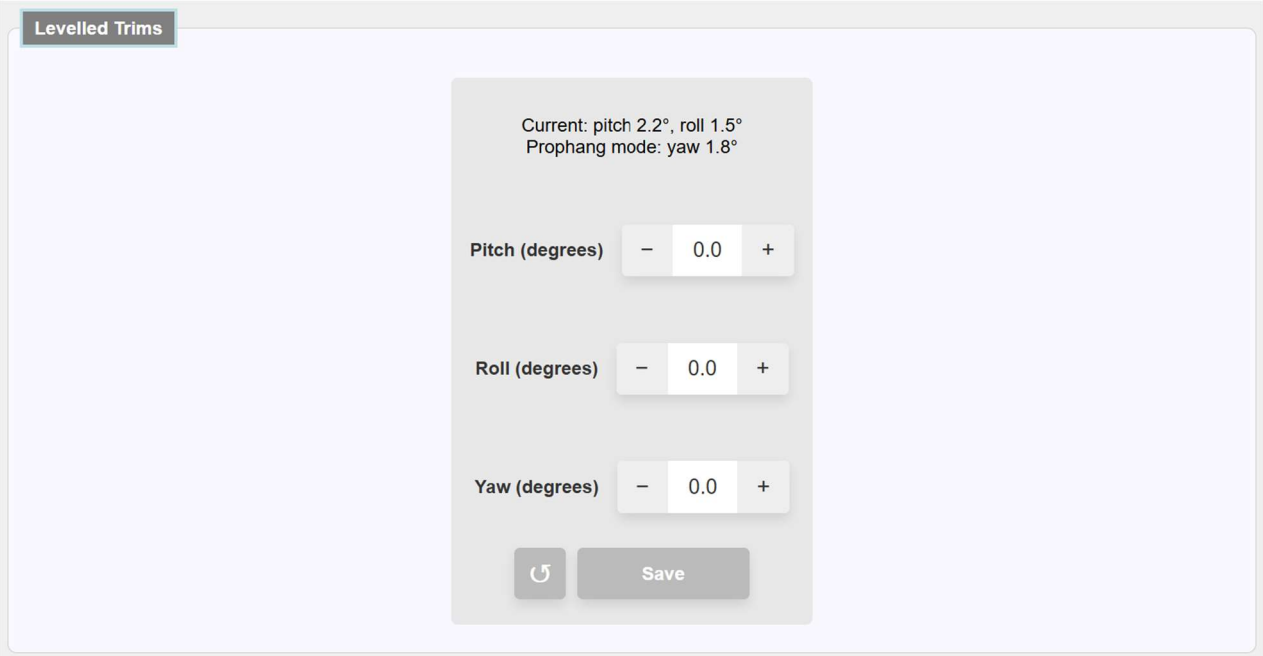
11.1.2.3 Self-Levelled Trim Configuration

If you find that your model does not maintain level flight when flying in self-levelled mode, then adjust the Levelled Trims values. Note these values are signed and depends on the orientation of you flight controller as to what signed value you require.

The easiest way to initially set these trims is to place you model on a flat/level surface or position the model in its flying attitude. Now use the ‘Current’ values as your trims but make them the opposite sign. i.e. with current pitch of 1.3 degrees, apply a pitch trim of -1.3. Yaw is only used when in prop hang mode i.e. when the models nose it vertical.

On editing the values, press save to store the new settings.

Figure 37 - Levelled Trims Form



11.1.2.4 Servo Trim Configuration

Note: Servo Trims are only to be used for centering the servo horn due to bad servo spline alignment. Always mechanically adjust control surfaces to be neutral with the servo horn at 90 degrees to the servo body. If you do not do this and use servo trims then you will reduce servo travel in one direction.

Servo trim depends on what model configuration you have selected, see section 7.3 Section 3: Model for definitions of servo numbers and what control surface they relate to.

On entering trim values, press save and you will instantly see the new trim value take effect on the servo.

Figure 38 - Servo Trims Form

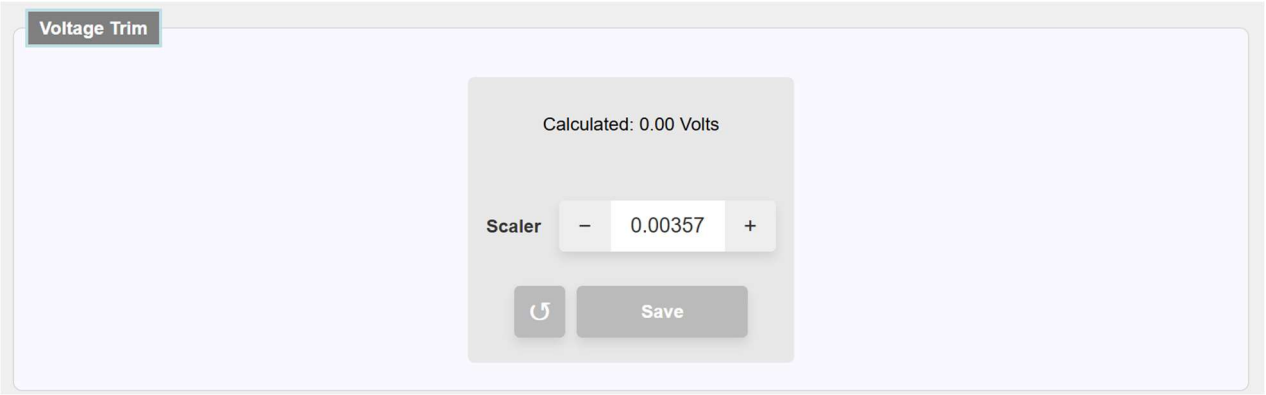


11.1.2.5 Battery Voltage Trims

If you are using USE_LOW_VOLTS_CUT_OFF to monitor the battery voltage, then you will likely need to calibrate the battery voltage due to resistor tolerances with the battery trim function.

On entering trim values, press save and you will instantly see the new trim value take effect on the calculated voltage. You can refresh the page to get the latest calculation, it is wise to do this as the voltage filtering may delay a true reading.

Figure 39 - Battery Voltage Trim



11.2 Reprogramming ESP32-S3 To Tune Parameters

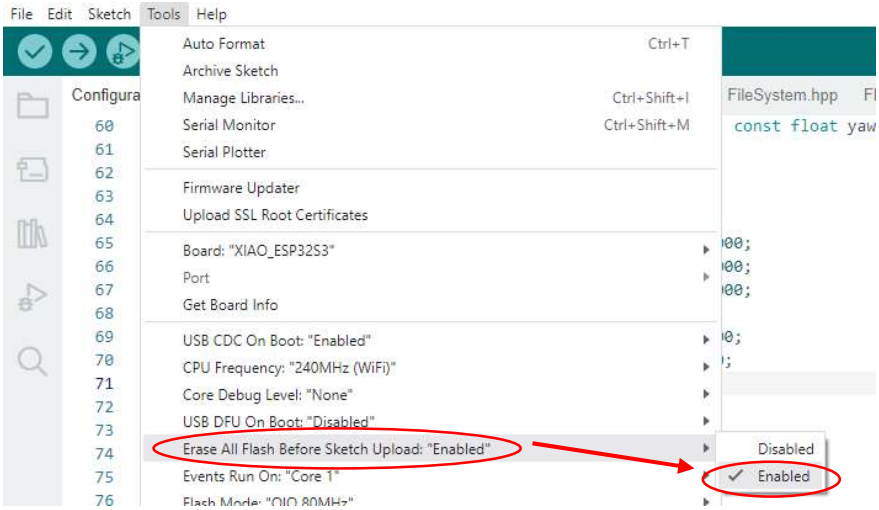
If you are unable to use the web configurator then you can still set all parameters by reprogramming the ESP32-S3. In Configurator.hpp are the default parameters, these can be manually edited and set to what you require:

Figure 40 - Default Parameters

```
//Default values for file system
//Rates
static constexpr  int32_t MAX_PITCH_RATE_DEGS_x100  = 20000;
static constexpr  int32_t MAX_ROLL_RATE_DEGS_x100   = 20000;
static constexpr  int32_t MAX_YAW_RATE_DEGS_x100    = 15000;
//Max angles allowed when in levelled mode (angles * 100)
static constexpr  int32_t MAX_PITCH_ANGLE_DEGS_x100 = 5500;
static constexpr  int32_t MAX_ROLL_ANGLE_DEGS_x100 = 5500;
//Trims
static constexpr  float LEVELLED_ROLL_TRIM        = 0.0f;
static constexpr  float LEVELLED_PITCH_TRIM       = 0.0f;
static constexpr  int32_t SERVO_TRIM              = 0;
//Gains
static constexpr  int32_t PITCH_P_GAIN              = 25;
static constexpr  int32_t PITCH_I_GAIN              = 50;
static constexpr  int32_t PITCH_D_GAIN              = 0;
static constexpr  int32_t PITCH_F_GAIN              = 18;
static constexpr  int32_t ROLL_P_GAIN               = 25;
static constexpr  int32_t ROLL_I_GAIN               = 50;
static constexpr  int32_t ROLL_D_GAIN               = 0;
static constexpr  int32_t ROLL_F_GAIN               = 18;
static constexpr  int32_t YAW_P_GAIN                = 25;
static constexpr  int32_t YAW_I_GAIN                = 50;
static constexpr  int32_t YAW_D_GAIN                = 0;
static constexpr  int32_t YAW_F_GAIN                = 10;
//Battery scaler
static constexpr float BATTERY_SCALER               = 0.00357f;
```

However, for any edits to take effect you will need to erase all flash before programming. This forces the file system to write the default parameters to a new file. In Arduino IDE make sure the following setting is enabled to force full flash erase, set Tools->Erase All Flash Before Sketch Upload->Enabled:

Figure 41 - Force Full Flash Erase



11.3 Rate Mode Tuning

Gyro rate mode is the most challenging to tune as you have three axis (pitch, roll and yaw) and four main parameters to tune per axis (PIDF).

Note: There is no gain attenuation based on throttle position as you will find on other flight controllers. To date I have found that conservative P with higher I-gains resolve this and gives excellent results. Adding D also can significantly improve the performance and reduces ‘bobble’ in flight by dampening P-gain, it also reduces/removes speed oscillations and ultimately allows slightly higher P-gains to be used – I have been amazed by how well D term works on servos, see my YouTube video on this.

Note: You will only need to use feed forward on servos.

To those who believe D-gain has no effect on servos you are partially correct. Servos will not respond to instantaneous pulses of D to speed the servo response, but the D term will prevent P-gain overshoot, softens ends of manoeuvres and disturbances and reduces/removed speed oscillations. This is of course with correct tuning.

Table 24 - Gyro Rate Mode Gains

Gain	Comment
xxx_F_GAIN	Feed forward does most of the work during a manoeuvre and the P and I terms are there to ensure that the degrees per second demand is met or maintained. With rate PID gains set to zero, set F-gain so you have at least 80% servo travel for full stick deflection when compared to pass-through mode. Note: As an example; When flying with PIF gains, perform a roll at maximum stick deflection and stop that roll quickly when level. If you notice bounce back (wings rolled back slightly) then F-gain is a little too high. If you notice or feel that the wings rolled more than you commanded, then feedforward is too low. Note: Multicopters generally do not need to use feed forward on motors.
xxx_P_GAIN	Increasing the P-gain results in faster corrections to disturbances. Too high a value will result in oscillation and possibly an uncontrollable situation – change to pass-through mode if you're ever in this situation. Airspeed will also affect this parameter and possible oscillations, so set this value conservatively for all axis. Note: Typical values that I have used vary between 45 and 100, but it will depend on control surface area, deflection and servo response times.
xxx_I_GAIN	I-gain gives 'stiffness' or 'heading hold' to each axis. Initially set the I gain to the same value as your P. However, do not be concerned to exceed the P-gain value. Note: Typical values that I have used vary between 100 and 300, but it will depend on control surface area, deflection and servo response times. **Note**: For fixed wing models, I-gain on yaw is only used when heading hold and/or prop hang mode are enabled.
xxx_D_GAIN	D-gain is used to stop P-gain overshoot and helps reduce oscillations due to the P term. Get your model flying with PIF initially, if you feel it is on the verge of oscillation, 'bobbles' in flight, or oscillates when at speed then add D-gain to the affected control axis to soften the settling of the control surface. I have found D-gains in the order of 250 to 1000 are effective. See my video on D-gains for fixed wing.

Note: Always set gains conservatively to avoid uncontrolled behaviour on first flights.

11.4 Self-Levelled Mode Tuning

As of PlainFlightController v1.1.0 There are no PIDF parameters to tune for levelled mode. It uses the rates and gains set for rate mode. However, a few parameters relate to its operation as described by 11.1.2.3 Self-Levelled Trim Configuration.

The algorithm implemented gives a very locked in flight feel as it uses the gyro rate controller and provides a snappy return to level if the pitch/roll stick is released quickly.

11.5 Heading Hold (Fixed Wing Only)

When heading hold is enabled you will have to tune yaw I-gain for it.

I-gain gives the 'heading hold' feature to the yaw axis. Typical values that I have used have been between 100 and 300. However, it will depend on control surface area, deflection, and servo response times.

Note: Heading hold has no effect on Multicopters as yaw I-gain is always enabled.

11.6 Model Gain Examples

The following sections give examples from some of my models to give you a rough idea of gains and gain ratios for different model types.

11.6.1 FLYING_WING with USE_DIFFERENTIAL_THROTTLE

~12” span (A4 paper size) twin motor (1102-10000kv) flying wing powered from 1s 650mAh lipo. Large elevon surfaces took a bit of tuning but found that D-gain dampened roll P-gain overshoot. 1.7g digital servos are running at SERVO_REFRESH_300HZ giving quickest possible update rate. Will prop-hnag hands off when using the prop-hang mode with differential thrust motors. With USE_LOW_VOLTS_CUT_OFF set to 3.4V limit and 3.3V cut off, flight times are around 8 minutes.



Note: Strongly advise use of 470uF 16V capacitors on power input of 1s BLHeli ESC’s to prevent voltage spikes getting to the ESP32-S3 processor – I learnt the hard way.

Table 25 - Example Of Settings Used By A4

Settings	Value
PLANE_FLYING_WING	true
USE_DIFFERENTIAL_THRUST	true
USE_PROP_HANG_MODE	true
PROP_HANG_TAIL_SITTER_MODE	true
USE_LOW_VOLTS_CUT_OFF	true
USE_250_DEGS_SECOND	true
Motor RefreshRate	IS_ONESHOT125
Servo RefreshRate	IS_300Hz
PITCH P	30
PITCH I	50
PITCH D	350
PITCH F	18
ROLL P	25
ROLL I	50
ROLL D	550
ROLL F	15
YAW P	50
YAW I	50
YAW D	550
YAW F	10
ROLL RATE (Degrees/Sec)	230
PITCH RATE (Degrees/Sec)	200
YAW RATE (Degrees/Sec)	150

11.6.2 PLANE_FULL_HOUSE

Hobbyking Rere Bear – spans 680mm and flown with 4s 2200mAh. This model is quick with Plain Flight GPS logging 149.4mph! Tuning this was easy as long as you understand its fast and needs small control movements & gyro corrections. To start with the elevator throw was reduced by one hole on the elevator

horn. The ailerons were reduced in throw by one hole on the aileron horns and by one hole on the servo horns to give slightly more sensible control throws.

This model is now incredibly locked in and allows flat out low speed passes with confidence. Launching is easy with minimal torque rolling from an under arm launch (unlike some friends Rare Bears). On this I am using battery and GPS telemetry via CRSF. Note if using GPS keep it away from any power wires and keep the GPS wires as short as possible to avoid interference. The only point to note with this model is that a value of 500D was added to stop some full speed oscillation on roll axis. This is a very impressive setup.



Table 26 – Example Of Settings Used By Rare Bear

Settings	Value
PLANE_FULL_HOUSE	true
USE_ONBOARD_NEOPIXEL	true
USE_250_DEGS_SECOND	true
Motor RefreshRate	IS_150Hz
Servo RefreshRate	IS_150Hz
PITCH P	20
PITCH I	100
PITCH D	0
PITCH F	18
ROLL P	16
ROLL I	100
ROLL D	250
ROLL F	14
YAW P	20
YAW I	100
YAW D	0
YAW F	40
ROLL RATE (Degrees/Sec)	225
PITCH RATE (Degrees/Sec)	160

11.6.3 PLANE_FULL_HOUSE

A 9.5” span all balsa sheet model with ailerons and elevator, powered by a 1102-10000kv brushless motor and 1s 300mAh LiHv. This model uses the Waveshare ESP32-S3 Tiny with built in RGB Neopixel LED. The MPU6050 IMU is a smaller version PCB than previously used and is located under the canopy.
Model has great vertical performance and with plainFlightController on board gives precision aerobatics.



Table 27 – Example Of Settings Used By Pea 9.5" Aerobat

Settings	Value
PLANE_FULL_HOUSE	true
USE_ONBOARD_NEOPIXEL	true
USE_250_DEGS_SECOND	true
Motor RefreshRate	IS_ONESHOT125
Servo RefreshRate	IS_100Hz
PITCH P	25
PITCH I	100
PITCH D	0
PITCH F	20
ROLL P	25
ROLL I	120
ROLL D	0
ROLL F	18
ROLL RATE (Degrees/Sec)	225
PITCH RATE (Degrees/Sec)	160

11.6.4 PLANE_FULL_HOUSE

This is an old FPV model I build back in 2015, balsa construction results in a stiff and responsive airframe. Has a non-symmetrical aerofoil that would cause it to ‘balloon’ when closing the throttle and when travelling at speed. Total of 10 degrees dihedral gives stability and allows rudder to steer the model. The high mounted motor can cause the nose to pitch down if throttle is applied quickly.

Adding PlainFlightController to this model has resolved any ‘ballooning’, rudder coupling and power pitching issues. This model is now so locked in it feels like a mini pattern ship to fly (it is stupidly overpowered with the Tmotor F40 too). 4-point rolls are fantastic, and with heading hold (and a lot of throttle) it will knife edge hands off.

Initially tuned with PIF gains only, with these settings I found that aileron corrections would cause the wing to ‘bobble’ when disturbed (not oscillate, just observe fast aileron corrections), not a problem and what you would expect for a flight controller. The yaw corrections would do the same for the rudder when travelling at speed. This bobbling was resolved by adding D-gain to both aileron and rudder which smooths any P-gain overshoot around the set point (neutral). This is now an incredibly smooth, locked in model to fly and is an absolute pleasure for aerobatics, which it was not originally designed for.

Four off 4g digital MG BlueArrow servos are running at SERVO_REFRESH_150HZ giving a reasonable response rate, and USING_ONESHOT125 to operate a BLHeli ESC.



Table 28 - Example Of Settings Used By OldFPV

Settings	Value
PLANE_FULL_HOUSE	true
USE_LOW_VOLTS_CUT_OFF	true
USE_FLAPS	true
USE_HEADING_HOLD	true
USE_250_DEGS_SECOND	true
Motor RefreshRate	IS_ONESHOT125
Servo RefreshRate	IS_150Hz
PITCH P	65
PITCH I	175
PITCH D	0
PITCH F	18
ROLL P	35
ROLL I	150
ROLL D	750
ROLL F	15
YAW P	40
YAW I	200
YAW D	600
YAW F	45
ROLL RATE (Degrees/Sec)	230
PITCH RATE (Degrees/Sec)	120
YAW RATE (Degrees/Sec)	100

11.6.5 PLANE_FULL_HOUSE

My original test bed during development of PlainFlightController was this donated and tatty Horizon Hobby Super Cub.



Now fitted with an AXI 2808 and some hastily added ailerons, it has made this into a nice slow fun-fly model. Nothing is straight or square on it and the wing flexes terribly when pulling manoeuvres. However, despite these problems its now a nice model that flies straight and true. Its party piece being hands off knife edge the length of our field when using heading hold, and very low slow knife edge (no heading hold). Flying this in self-levelled mode using only throttle and rudder controls to perform touch and goes has also been quite good fun. With heading hold this model will hold itself in prop-hang hands off, though lack of airflow over ailerons does mean that it drifts off after several seconds.

The PI gains are quite high on this model for 2 reasons, the original servos are quite slow to respond (even though SERVO_REFRESH_150HZ is set) and there is so much flex in the airframe that control corrections are absorbed before the model responds. To date I have not found D-gain to give any beneficial effect on this model and believe this is because of the flex in the model.

Settings	Value
PLANE_FULL_HOUSE	true
USE_FLAPS	true
USE_HEADING_HOLD	true
USE_PROP_HANG_MODE	true
USE_250_DEGS_SECOND	true
Motor RefreshRate	IS_150Hz
Servo RefreshRate	IS_150Hz
PITCH P	100
PITCH I	300
PITCH D	0
PITCH F	18
ROLL P	75
ROLL I	150
ROLL D	0
ROLL F	25
YAW P	80
YAW I	300
YAW D	0
YAW F	45
ROLL RATE (Degrees/Sec)	180
PITCH RATE (Degrees/Sec)	180
YAW RATE (Degrees/Sec)	100

11.6.6 PLANE_ADVANCED_RUDDER_ELEVATOR

Pip is a simple 16” span rudder elevator model. Its non-symmetrical aerofoil makes it quite buoyant and wants to climb when throttle is quickly reduced (effective down thrust removed). Its light weight also makes it susceptible to weather conditions.

Adding PlainFlightController instantly removes any pitch issues related to speed and down thrust and also makes it fly rock solid in windy conditions.

Being rudder elevator, it was found that the wings would rock a little due to corrections in windy conditions. This just looked odd and poorly tuned. As a result, PLANE_ADVANCED_RUDDER_ELEVATOR was implemented to allow separate PIDF controls for roll and pitch to be applied to rudder. In effect one cancels the other out and gives a more stable flight. However, it is a little harder to tune as result. PLANE_ADVANCED_RUDDER_ELEVATOR also has benefits for stall turns as yaw PIDF controller is directly controlling yaw rather than roll PIDF controlling yaw which would lead to roll I-gain build up and a roll response when exiting the stall turn.



Settings	Value
PLANE_ADVANCED_RUDDER_ELEVATOR	true
USE_LOW_VOLTS_CUT_OFF	true
USE_HEADING_HOLD	true
USE_250_DEGS_SECOND	true
Motor RefreshRate	IS_ONESHOT125
Servo RefreshRate	IS_150Hz
PITCH P	65
PITCH I	175
PITCH D	650
PITCH F	18
ROLL P	35
ROLL I	150
ROLL D	750
ROLL F	15
YAW P	40
YAW I	150
YAW D	600
YAW F	45
ROLL RATE (Degrees/Sec)	220
PITCH RATE (Degrees/Sec)	180
YAW RATE (Degrees/Sec)	100

11.6.7 QUAD_X_COPTER

OK don't expect any PlainFlightController multicopters to fly as good as BetaFlight. However, be surprised at how well your home built quad frame can fly, even aerobatics.



Consider the gains given to be a basic tune for a low powered quad it can certainly be improved. However, this model happily loops, rolls and stall turns with the settings given.

D-gain was crucial to getting any sense out of making it fly nicely. Add I-gain until each axis feel locked in, be slightly sparing with P-gain. Add D-gain to overcome any 'bobbles'.

Settings	Value
QUAD_X_COPTER	true
USE_LOW_VOLTS_CUT_OFF	true
USE_HEADING_HOLD	true
USE_250_DEGS_SECOND	false
Motor RefreshRate	IS_ONESHOT125
PITCH P	20
PITCH I	150
PITCH D	950
PITCH F	0
ROLL P	20
ROLL I	150
ROLL D	950
ROLL F	0
YAW P	45
YAW I	150
YAW D	250
YAW F	0
ROLL RATE (Degrees/Sec)	360
PITCH RATE (Degrees/Sec)	360
YAW RATE (Degrees/Sec)	200

12. Pre-Flight Checks

Strongly recommend that the following pre-flight check are carried out as a minimum. Not doing so may result in the loss of your model. Perform any additional checks you feel are necessary or wise to ensure safety and reliability of your model.

Note: It is assumed that you have experience in using and tuning flight controllers and understand PIDF control systems. If you are not confident seek advice or training.

Caution	Check	Comments
	Ensure servos are centred and model control surfaces are mechanically trimmed to neutral positions.	Set servo horns to correct spline location. Use TRIM_SERVOx to centre servos before flying.
	Ensure flight controller is firmly attached to model and in correct orientation.	Attach with double sided foam tape or Velcro. Make sure no lose items can rub or strike the flight controller when in flight. Make sure all wired connections are secure.
	Ensure pitch, roll, and yaw gyro corrections operate in the correct sense when in rate mode.	Use REVERSE_SERVO_x to reverse pitch, roll, yaw corrections for rate mode.
	Ensure pitch, roll, and yaw operate in the correct sense to Tx commands.	If gyro and self-levelled corrections work correctly but Tx commands are the wrong sense, then reverse direction of affected channel(s) in your transmitter software.
	Ensure power supply is sufficient for increased current demands of servos.	Move model around rapidly in rate mode for 10 seconds to ensure power supply does not brown out. With a helper; repeat rapid movements whilst pulsing throttle between low and full to ensure motor load does not affect power supply.
	Set rate mode gains to sensible values.	Default gains given may not be appropriate for you model. Set gains so you can just see corrections occurring for first flight.
	For first flight, fly model in pass through mode until you are confident.	For first tuning flights only change out of pass-through mode when you have sufficient height to try and recover the model in case gains are set too high tuned.

13. Disclaimer & License

Do not expect this software to be comparable to or outperform other more established flight controller projects such as ArduPilot, inav, betaFlight etc. This code shall also be considered as highly experimental and is not designed or written to any safety critical, or mission critical standards.

It is given/shared for free with the knowledge and understanding that this open-source flight controller software is only for small hobby based electrically powered model aircraft, or other small hobby radio-controlled vehicles. It is intended to be used or modified to suit your needs for small models and is NOT to be used on any manned vehicles.

The author(s) shall not be held responsible or accountable for any damage, injury or loss that may be inflicted or incurred as a result of the use or miss use of this software. You use and/or modify this software entirely at your own risk and it must be used within accordance of your country's laws and/or regulations.

By using this software, or any part of this software you agree to GPL-3.0 license (http://www.gnu.org/licences/).